



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по практической работе №3

по дисциплине «Проектирование и разработка мобильных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л. А.

Проверил:

старший преподаватель кафедры
МОСИТ

Шешуков Л. С.

МОСКВА 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 Передача сложных объектов между активностями.....	3
1.2 BackStack.....	7
1.3 Контейнеры.....	10
1.3.1 LinearLayout.....	10
1.3.2 RelativeLayout.....	11
1.3.3 FrameLayout.....	12
1.3.4 TableLayout.....	13
1.3.5 GridLayout.....	14
1.3.6 ConstraintLayout.....	14
1.4 Ресурсы в Android.....	16
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	24
2.1 Создание проекта.....	24
2.2 ConstraintLayout.....	25
2.3 LinearLayout.....	26
2.4 RelativeLayout.....	27
2.5 FrameLayout.....	30
2.6 Добавление ресурсных файлов.....	32
2.7 Изменение темы.....	34
2.8 Локализация.....	37
ЗАКЛЮЧЕНИЕ.....	44

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1.1 Передача сложных объектов между активностями

В предыдущей практике передавались простые данные – числа, строки. Но также можно передавать более сложные данные.

Передача сложных объектов между активностями в Android осуществляется с помощью механизма сериализации и передачи данных через Intent.

Сериализация — это процесс преобразования объекта в последовательность байтов, которая может быть сохранена в памяти устройства или передана по сети, а затем восстановлена обратно в объект.

Для передачи сложных объектов между активностями необходимо, чтобы класс объекта имплементировал интерфейс Serializable или Parcelable. Оба интерфейса предоставляют способы сериализации объекта, но Parcelable часто используется в Android, так как он более эффективен в производительности.

В Android Studio (как и в Java) интерфейс Serializable используется для маркировки классов, объекты которых могут быть сериализованы. После этого объект может быть восстановлен (десериализован) обратно.

Основные особенности Serializable в Android:

- маркерный интерфейс: как и в Java, Serializable не содержит методов. Он просто указывает, что объекты класса могут быть сериализованы;
- использование в Android: передача данных между Activity через Intent. Сохранение состояния объекта в Bundle. Сохранение данных в файл или базу данных;
- простота использования: для реализации достаточно указать, что класс реализует интерфейс Serializable.

Интерфейс Parcelable в Android используется для сериализации объектов, чтобы их можно было передавать между компонентами Android, такими как Activity, Fragment или Service. В отличие от Serializable, Parcelable более

эффективен и оптимизирован для Android, так как не использует reflection и работает быстрее.

Основные особенности Parcelable:

- Высокая производительность: Parcelable работает быстрее, чем Serializable, так как процесс сериализации и десериализации реализуется вручную, без использования reflection;
- Ручная реализация: Для использования Parcelable необходимо вручную описать процесс записи (сериализации) и чтения (десериализации) объекта.;
- Использование в Android: Parcelable часто используется для передачи данных через Intent или сохранения состояния в Bundle.

Методы интерфейса Parcelable:

- `describeContents()`: Возвращает специальные флаги, которые описывают содержимое объекта. Обычно возвращает 0, если нет специальных требований;
- `writeToParcel(Parcel dest, int flags)`: Записывает данные объекта в Parcel. В этом методе вы указываете, как именно нужно сериализовать объект;
- `Parcelable.Creator<T>`: Статическое поле, которое используется для создания объекта из Parcel. Оно содержит методы `createFromParcel()` и `newArray()`.

Процесс передачи объекта между активностями включает следующие шаги:

- создание объекта для передачи: сначала необходимо создать экземпляр класса, который вы хотите передать между активностями;
- имплементация интерфейса Parcelable (или Serializable): если класс объекта не имплементирует Parcelable, то необходимо добавить соответствующие методы чтения и записи для сериализации и десериализации данных объекта;
- упаковка объекта в Intent: для передачи объекта между активностями необходимо создать новый Intent и использовать его методы `putExtra()` или `putParcelableExtra()`, чтобы добавить объект в Intent.
- Сначала создадим класс, который будем передавать. Например, `MyObject`.

Для создания класса нажимает правой кнопкой мыши на "java" → "New" → "Java Class" (рисунок 1.1.1). После выбора вводим название класса (рисунок 1.1.2).

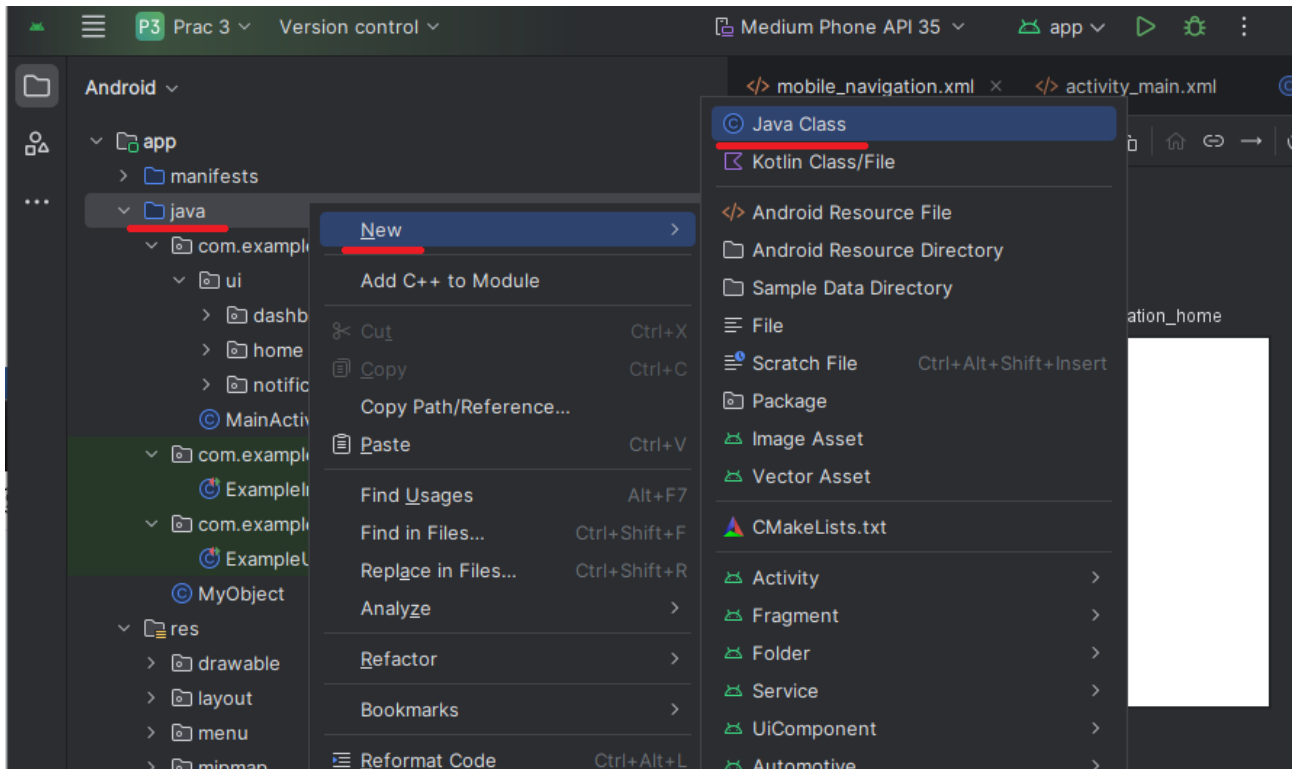


Рисунок 1.1.1 - Создание нового класса в Android Studio

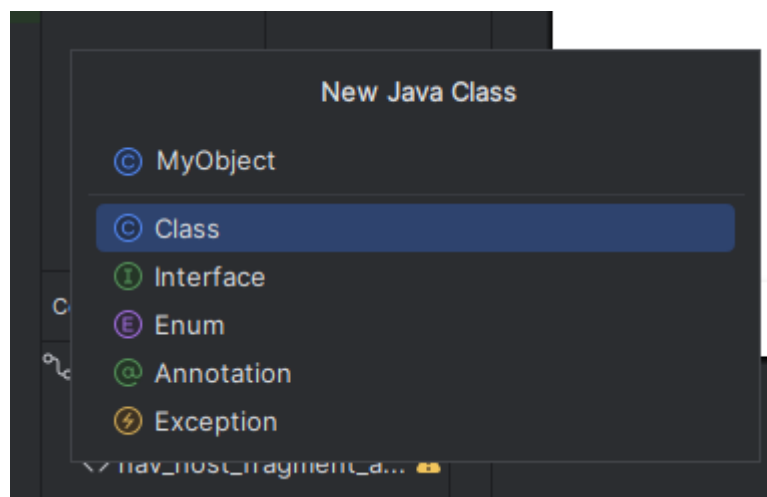


Рисунок 1.1.2 – Именованное нового класса

После создания класса необходимо описать его. Для этого заходим в класс `MyObject`, объявляем внутри него переменные и необходимые методы (рисунок 1.1.3).

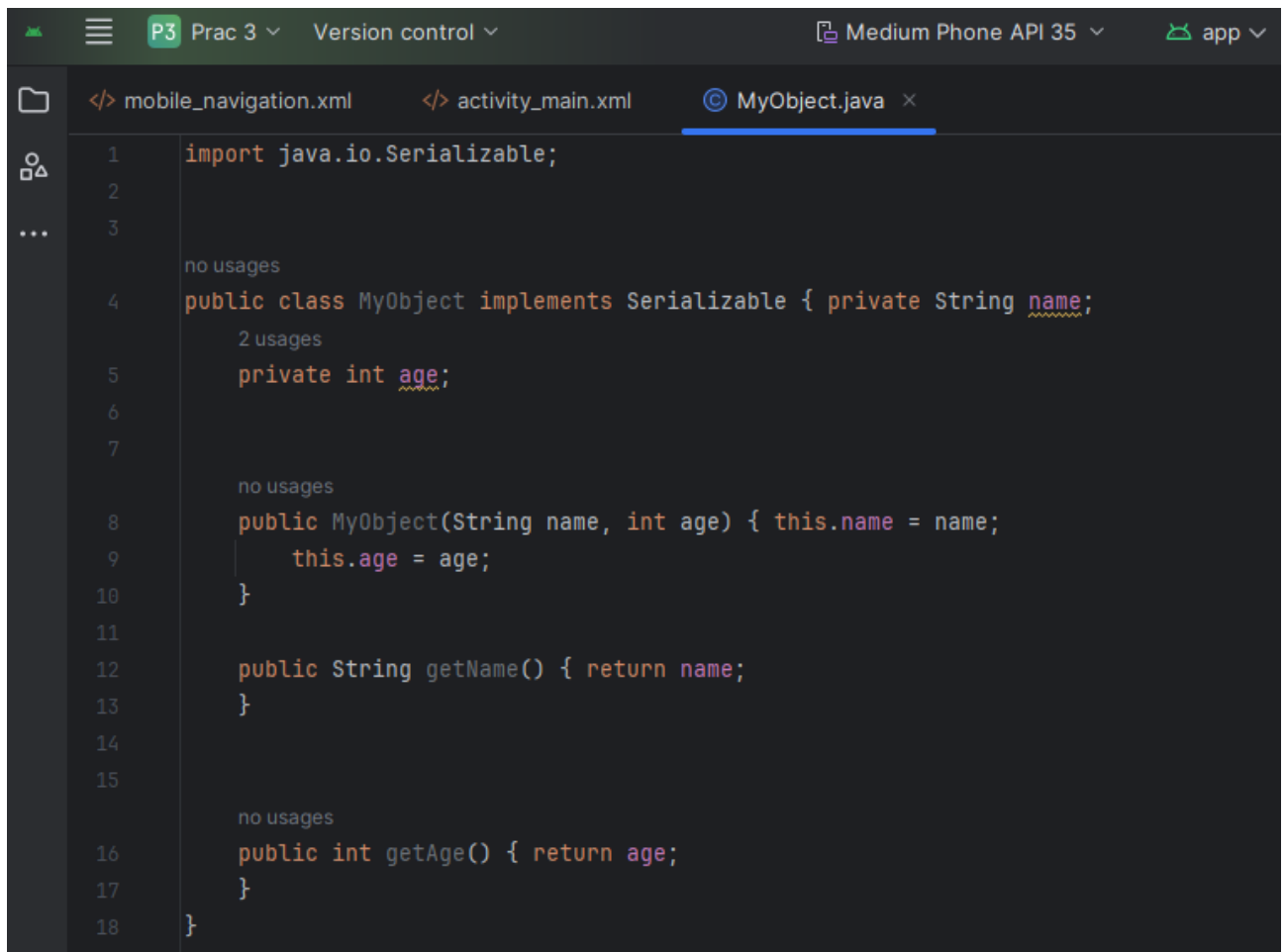


Рисунок 1.1.3 – Описание класса MyObject

Затем создадим объект этого класса в активности (рисунок 1.1.4).



Рисунок 1.1.4 – Создание объекта класса MyObject

После чего необходимо извлечь объект из целевой активности (рисунок 1.1.5).

```

12 </> public class NewActivity extends AppCompatActivity {
13
14     @Override
15     protected void onCreate(Bundle savedInstanceState) {
16         super.onCreate(savedInstanceState);
17         EdgeToEdge.enable( $this$enableEdgeToEdge: this);
18         setContentView(R.layout.activity_new);
19         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
20             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
21             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
22             return insets;
23         });
24
25         TextView messageText = new TextView( context: this);
26         messageText.setTextSize(26);
27         messageText.setPadding( left: 16, top: 16, right: 16, bottom: 16);
28
29         MyObject myObjectInput = (MyObject) getIntent().getSerializableExtra( name: "myObject");
30         if (myObjectInput != null){
31             messageText.setText("Name: " + myObjectInput.getName() + "\nAge" + myObjectInput.getAge());
32         }
33     }

```

Рисунок 1.1.5 – Извлечение объекта класса MyObject в целевой активности

Таким образом, осуществляется передача сложных объектов между разными активностями.

1.2 BackStack

BackStack в Android — это концепция, используемая для управления навигацией между различными экранами в приложении. Этот механизм можно представить как стек, где каждый элемент представляет собой активность или фрагмент, к которому пользователь может вернуться, нажав кнопку «Назад». Например, когда пользователь открывает новую активность, она помещается в вершину стека. При нажатии кнопки «Назад» текущая активность удаляется из стека, и пользователь возвращается к предыдущей активности, которая теперь находится на вершине стека.

Android предоставляется возможность программно манипулировать BackStack, что позволяет настраивать поведение навигации в своих приложениях. Так можно программно удалить определённые активности из стека или очистить весь стек, чтобы предотвратить возвращение пользователя к предыдущим экранам.

В Android, BackStack (стек возврата) используется для управления навигацией между фрагментами и активностями. Когда разработчик добавляет фрагмент или активность в стек, пользователь может вернуться к предыдущему экрану, нажав кнопку "Назад". Флаги в BackStack позволяют управлять поведением стека, например, очищать его, заменять фрагменты или добавлять новые записи.

Наиболее популярные флаги, которые используются при работе с BackStack, это флаги для метода `addToBackStack()` и флаги для транзакций фрагментов (`FragmentManager.Transaction`). Вот основные из них:

- `FragmentManager.POP_BACK_STACK_INCLUSIVE`: удаляет из стека все транзакции до указанной (включая саму указанную транзакцию);
- `FragmentManager.POP_BACK_STACK_IMMEDIATE`: немедленно выполняет удаление из стека, не дожидаясь завершения текущей операции;
- `Intent.FLAG_ACTIVITY_CLEAR_TOP`: если активность, которую вы запускаете, уже существует в стеке, все активности поверх нее будут закрыты, и она станет верхней в стеке;
- `Intent.FLAG_ACTIVITY_NEW_TASK`: запускает активность в новой задаче (`Task`). Это полезно, например, при запуске активности из уведомления;
- `Intent.FLAG_ACTIVITY_SINGLE_TOP`: если активность уже находится на вершине стека, она не будет создана заново. Вместо этого будет вызван метод `onNewIntent()`;
- `Intent.FLAG_ACTIVITY_CLEAR_TASK`: очищает всю задачу (`Task`) перед запуском новой активности. Все существующие активности в задаче будут удалены;
- `Intent.FLAG_ACTIVITY_NO_HISTORY`: активность, запущенная с этим флагом, не будет добавлена в BackStack. После закрытия она не сможет быть восстановлена.

Предположим, что в `NewActivity` пользователь выполняет некоторые действия, и необходимо, чтобы при возвращении в `MainActivity`, `NewActivity`

была удалена из BackStack. В этом случае можно использовать флаг `Intent.FLAG_ACTIVITY_CLEAR_TOP` (рисунок 1.2.1).

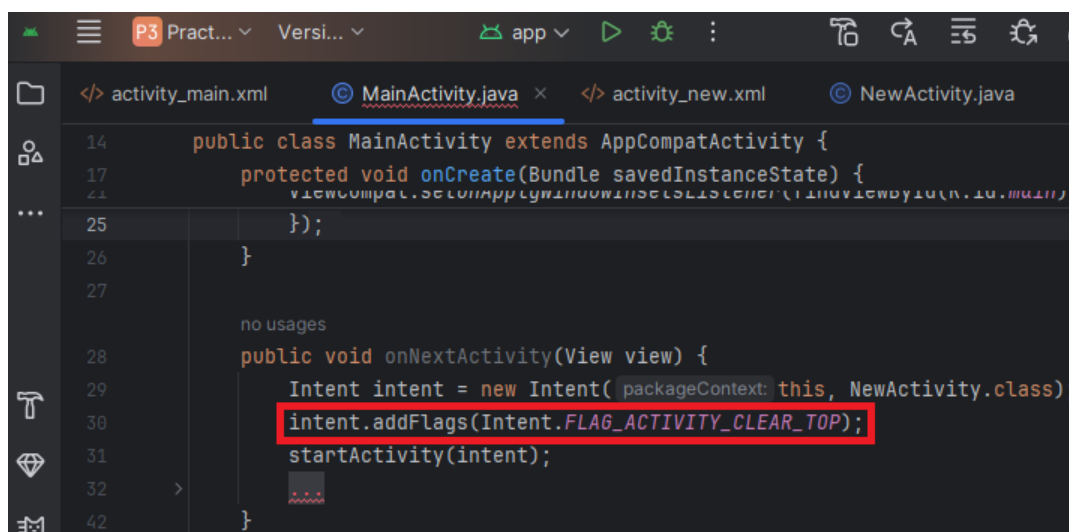


Рисунок 1.2.1 – Пример использования флага `Intent.FLAG_ACTIVITY_CLEAR_TOP`

Добавление флага `FLAG_ACTIVITY_CLEAR_TOP` будет указывать системе, что при переходе к MainActivity, все активности, находящиеся над MainActivity в стеке, должны быть удалены. Таким образом, NewActivity будет удалена из стека, и при нажатии кнопки «Назад» в MainActivity, пользователь выйдет из приложения, а не вернется к NewActivity.

С помощью метода `finish()` можно завершить работу активности (рисунок 1.2.2). Если приложение состоит из одной активности, то этого делать не следует, так как система сама завершит работу приложения. Если же приложение содержит несколько активностей, между которыми нужно переключаться, то данный метод позволяет экономить ресурсы.

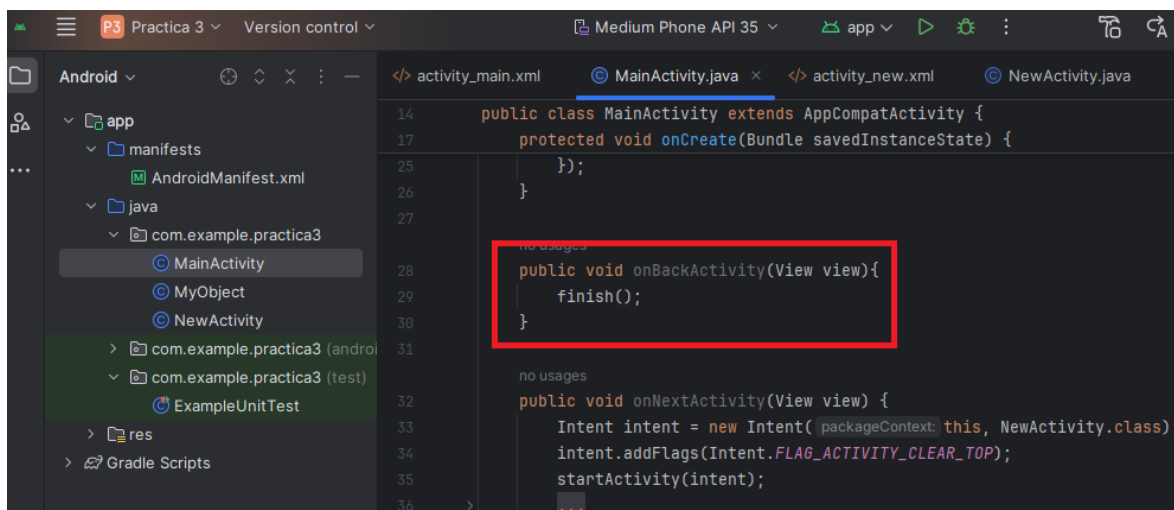


Рисунок 1.2.2 – Пример использования метода `finish()`

1.3 Контейнеры

Контейнер компоновки в Android — это специализированный объект, который используется для управления расположением элементов пользовательского интерфейса (таких как кнопки, текстовые поля, изображения и другие элементы) на экране устройства. Эти контейнеры обеспечивают структуру и организацию элементов пользовательского интерфейса в приложении Android.

В основе работы контейнера компоновки лежит концепция управления размещением и размером дочерних элементов в соответствии с определенными правилами. Контейнеры компоновки определяют, как элементы интерфейса должны быть расположены относительно друг друга и как они должны вести себя при изменении размеров экрана или ориентации устройства. Это особенно важно в Android, где приложения используются на широком спектре устройств с различными размерами и разрешениями экранов.

Существует несколько типов контейнеров компоновки в Android, каждый из которых предлагает различные стратегии для расположения элементов.

1.3.1 LinearLayout

Упорядочивает элементы в линейном порядке, вертикально или горизонтально. Это простой и интуитивно понятный способ компоновки, где элементы размещаются один за другим. `LinearLayout` поддерживает такое свойство, как вес элемента, которое передается атрибутом `android:layout_weight`. Это свойство принимает значение, указывающее, какую часть оставшегося свободного места контейнера по отношению к другим объектам займет данный элемент. Например, если один элемент у нас будет иметь для свойства `android:layout_weight` значение 1, а другой – значение 3, то в сумме они дадут 4, поэтому первый элемент будет занимать 1/4 оставшегося пространства, а второй - 3/4 (рисунки 1.3.1.1 – 1.3.1.2).

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:text="Hello"
        android:background="#e0e0e0"
        android:layout_weight="1"
        android:textSize="30sp" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:text="World"
        android:background="#bdbdbd"
        android:layout_weight="3"
        android:textSize="30sp" />
</LinearLayout>

```

Рисунок 1.3.1.1 – Программная часть использования атрибута `android:layout_weight`

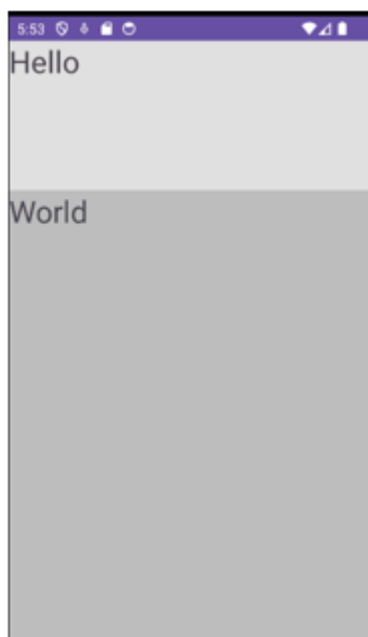


Рисунок 1.3.1.2 – Графическая часть использования атрибута `android:layout_weight`

1.3.2 RelativeLayout

Позволяет располагать элементы относительно друг друга или относительно родительского контейнера. Это обеспечивает большую гибкость в определении местоположения и размера элементов (рисунки 1.3.2.1 – 1.3.2.2).

```

<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView android:text="Left Top"
        android:layout_height="wrap_content"
        android:layout_width="wrap_content"
        android:textSize="30sp"
        android:layout_alignParentLeft="true"
        android:layout_alignParentTop="true" />

    <TextView android:text="Right Bottom"
        android:layout_height="wrap_content"
        android:layout_width="wrap_content"
        android:textSize="30sp"
        android:layout_alignParentRight="true"
        android:layout_centerInParent="true" />

</RelativeLayout>

```

Рисунок 1.3.2.1 – Программная часть использования RelativeLayout



Рисунок 1.3.2.1 – Графическая часть использования RelativeLayout

1.3.3 FrameLayout

Предназначен для размещения одного элемента поверх другого. Это может быть полезно для создания сложных перекрывающихся интерфейсов, например, для отображения загрузочных индикаторов или для создания пользовательских компонентов интерфейса (рисунки 1.3.3.1 - 1.3.3.2).

```

<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="First Text!"
        android:textSize="30sp"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Second Text"
        android:textSize="30sp"
        android:layout_marginTop="70dp"/>

```

Рисунок 1.3.3.1 – Программная часть использования FrameLayout

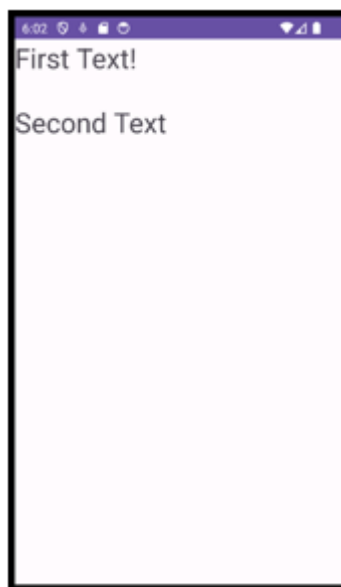


Рисунок 1.3.3.2 – Графическая часть использования FrameLayout

1.3.4 TableLayout

Контейнер, который позволяет организовывать элементы интерфейса (виджеты) в виде таблицы с строками и столбцами. Он похож на HTML-таблицу, где можно добавлять строки (TableRow) и размещать в них элементы. TableLayout является подклассом LinearLayout, поэтому он наследует его свойства, но добавляет возможность управления строками и столбцами. Структурирует элементы управления в виде таблицы по столбцам и строкам (рисунок 1.3.4.1).

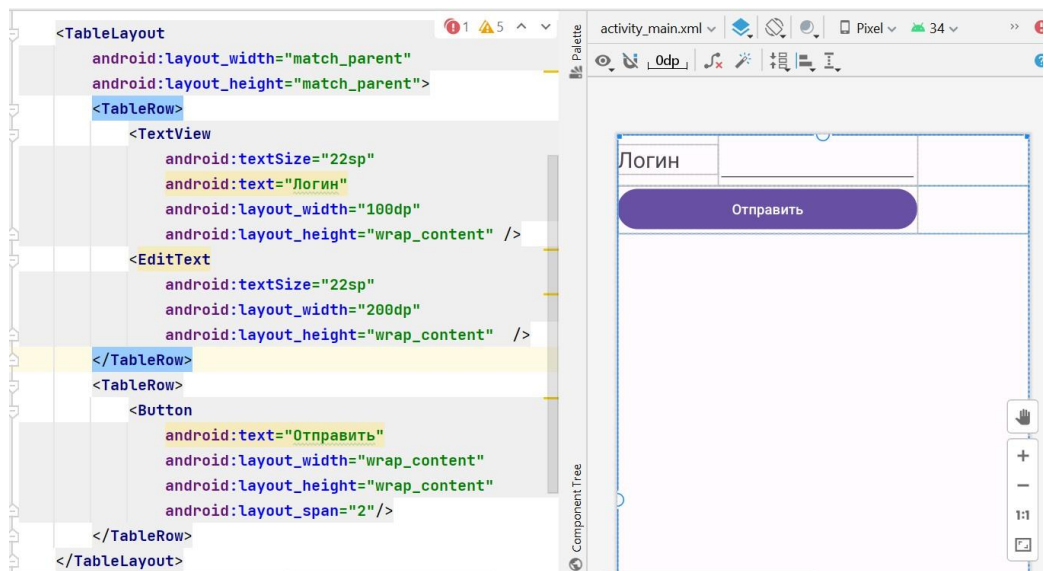


Рисунок 1.3.4.1 – Программная и графическая часть использования TableLayout

1.3.5 GridLayout

Представляет еще один контейнер, который позволяет создавать табличные представления. GridLayout состоит из коллекции строк, каждая из которых состоит из отдельных ячеек (рисунок 1.3.5.1).

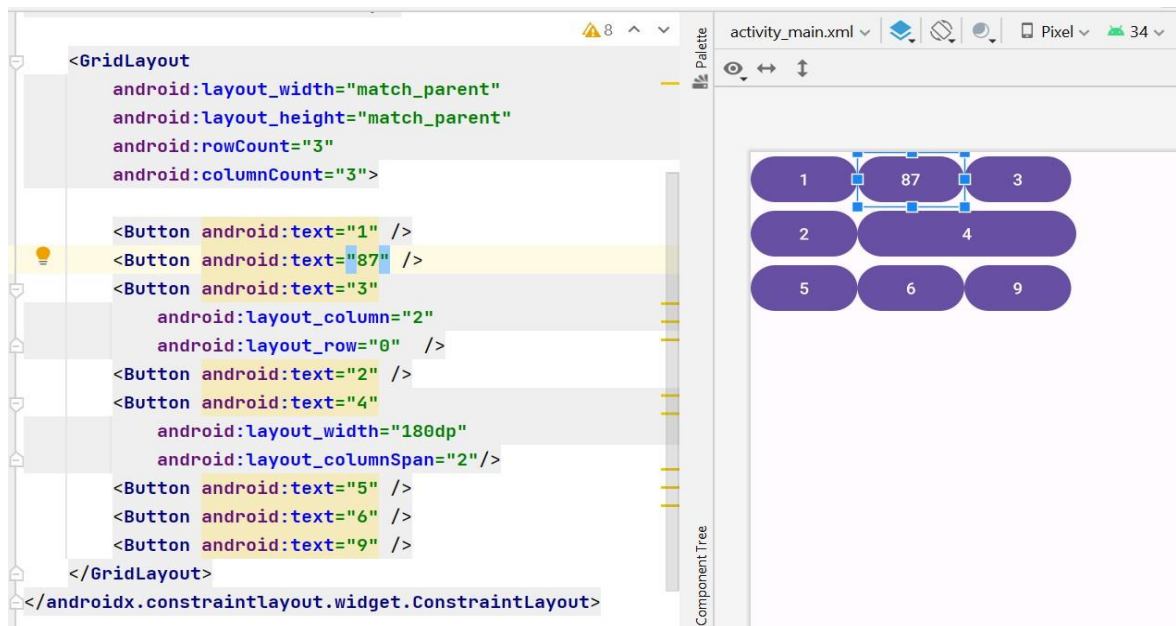


Рисунок 1.3.5.1 – Программная и графическая часть использования GridLayout

1.3.6 ConstraintLayout

Относительно новый тип контейнера компоновки, который позволяет создавать сложные и гибкие интерфейсы с помощью набора ограничений для

определения положения и размера виджетов. По умолчанию при открытии Activity все элементы располагаются в этом контейнере (рисунок 1.3.6.1).

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <EditText
        android:id="@+id/editText"
        android:layout_width="180dp"
        android:layout_height="wrap_content"
        android:hint="Введите ваш логин"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Войти"
        app:layout_constraintLeft_toRightOf="@id/editText"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Рисунок 1.3.6.1 – Программная часть использования ConstraintLayout

Корневой элемент содержит определение используемых пространств имен XML. В коде по умолчанию в ConstraintLayout мы можем увидеть такие атрибуты, как xmlns:android, xmlns:app, xmlns:tools (рисунок 1.3.6.2).

```
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
```

П Название

Рисунок 1.3.6.2 – Список атрибутов по умолчанию в ConstraintLayout

Каждое пространство имен задается следующим образом: xmlns:префикс="название_ресурса".

Через префикс мы сможем ссылаться на функциональность этого пространства имен.

Каждое пространство имен определяет некоторую функциональность, которая используется в приложении, например, предоставляют теги и атрибуты, которые необходимы для построения приложения. Далее следует описание наиболее распространённых пространств имён:

- xmlns:android="http://schemas.android.com/apk/res/android": содержит

основные атрибуты, которые предоставляются платформой Android, применяются

в элементах управления и определяют их визуальные свойства (например, размер, позиционирование);

- `xmlns:app="http://schemas.android.com/apk/res-auto"`: содержит атрибуты, которые определены в рамках приложения;
- `xmlns:tools="http://schemas.android.com/tools"`: применяется для работы с режиме дизайнера в Android Studio.

Для организации элементов внутри контейнера используются параметры разметки. Для их задания в файле `xml` используются атрибуты, которые начинаются с префикса `layout_`. В частности, к таким параметрам относятся атрибуты `layout_height` и `layout_width`, которые используются для установки размеров. Данные атрибуты могут иметь следующие значения:

- `match_parent`: установка данного значения позволяет растянуть элемент по всей ширине или высоте контейнера. Данное значение применяется ко всем контейнерам, кроме `ConstraintLayout`. `match_parent` также не рекомендуется применять к элементам внутри `ConstraintLayout`. Вместо этого можно использовать значение `0dp`, чтобы растянуть элемент по горизонтали или вертикали;
- `wrap_content`: устанавливает те значения для ширины или высоты, которые необходимы, чтобы разместить на экране содержимое элемента;
- точные значения.

1.4 Ресурсы в Android

Ресурсы в Android — это специальные элементы, используемые для хранения различных типов данных, отделённых от кода приложения. Эти ресурсы включают в себя такие элементы, как строки, цвета, изображения, макеты и другие. Важной особенностью ресурсов является их удобство управления и изменения без необходимости перекомпиляции всего приложения.

Строковые ресурсы используются для хранения текстовой информации, такой как сообщения, названия кнопок и прочее (рисунок 1.4.1). Хранение строк в отдельных ресурсах облегчает локализацию приложений, позволяя легко заменять текст на различные языки без изменения исходного кода.

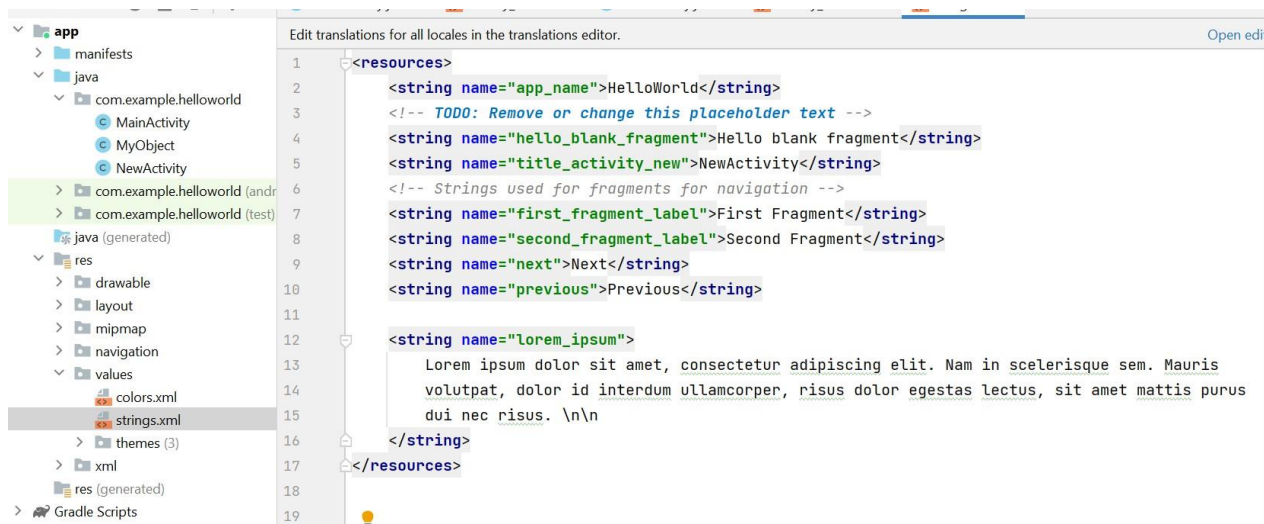


Рисунок 1.4.1 – Файл с текстовыми ресурсами

Цветовые ресурсы используются для определения цветовой палитры приложения, включая цвета фона, текста, кнопок и других элементов интерфейса (рисунок 1.4.2). Хранение цветов в отдельных ресурсах позволяет унифицировать и централизовать управление цветовой схемой и упрощая процесс дизайна. Это также упрощает изменения цветовой схемы для разных тем или режимов (например, дневного и ночного), позволяя менять цвета во всем приложении без необходимости редактирования каждого элемента вручную.

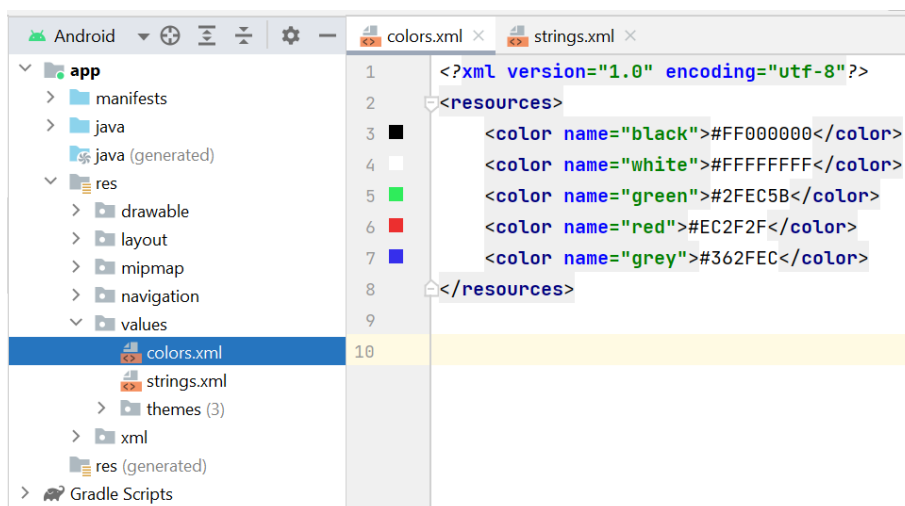


Рисунок 1.4.2 – Файл с цветовыми ресурсами

Оба ресурса находятся в пакете `values` и создаются по одному шаблону: сначала указывается тип данных, например `color` или `string`, затем в поле `name` указывается название ресурса и то значение, которое он будет содержать.

Изображения и графика, обычно хранятся в пакете `drawable`, в форматах PNG и JPEG, с остальными в Android Studio могут возникать ошибки. Они используются для улучшения визуальной составляющей приложения. Размещение графических ресурсов в отдельной папке позволяет легко обновлять и изменять изображения, не затрагивая остальную часть приложения.

Для добавления в проект в папку `res/drawable` какого-нибудь файла изображения необходимо скопировать на жестком диске какой-нибудь файл с расширением `png` или `jpg` и вставим его в папку `res/drawable` (для копирования в проект используется простой Copy-Paste).

Далее нам будет предложено выбрать папку - `drawable` или `drawable-24` (рисунок 1.4.3).

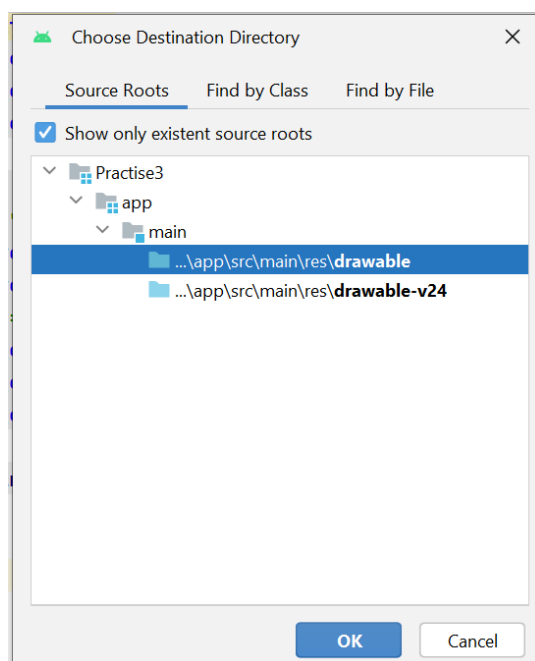


Рисунок 1.4.3 – Добавление в проект файла в формате .png/jpg

Для добавления обычных файлов изображений была выбрана папка `drawable`.

При копировании файла нам будет предложено установить для него новое имя (рисунок 1.4.4).

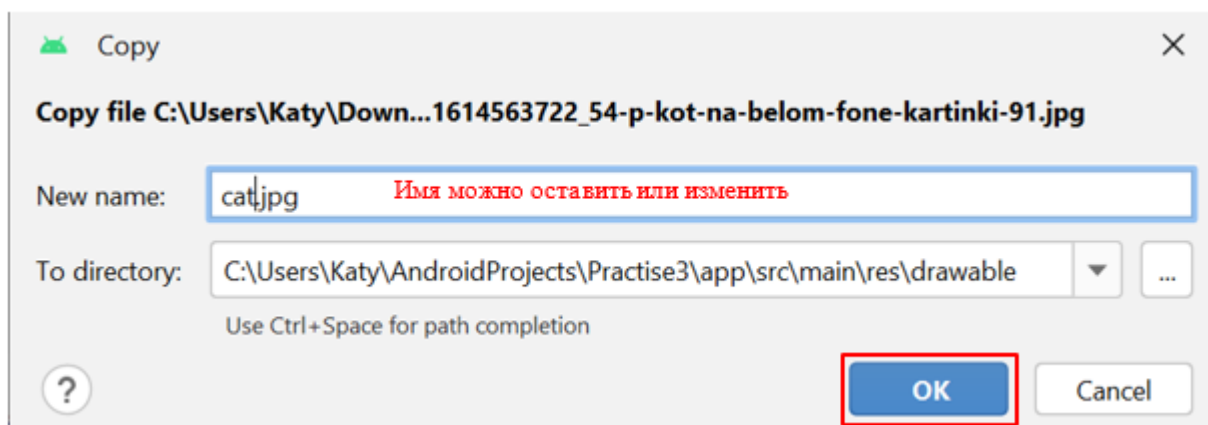


Рисунок 1.4.4 – Именованное копируемого файла

После этого в папку drawable будет добавлен выбранный нами файл изображения. Для отображения файла в ImageView у элемента устанавливается атрибут android:src.

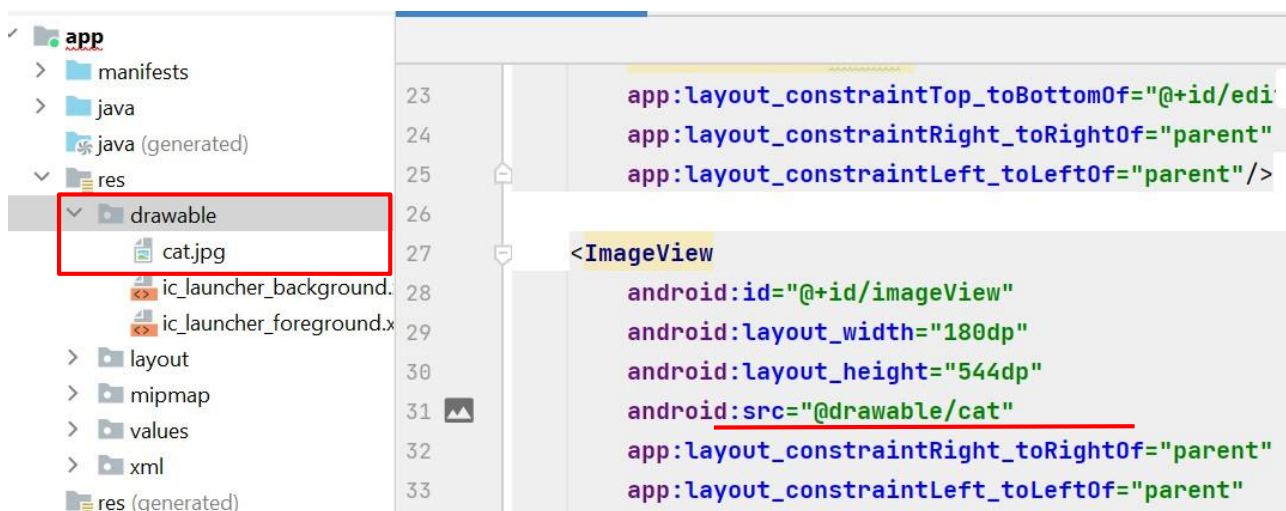


Рисунок 1.4.5 – Отображение в коде скопированного файла

Макеты описывают структуру пользовательского интерфейса приложения, определяют расположение элементов, таких как кнопки, текстовые поля, и другие виджеты. Хранятся такие макеты в пакете layout.

Ресурсы позволяют детальнее индивидуализировать приложение, поменяв тему или локализовать его для конкретной страны или региона.

Для смены темы необходимо в пакете values перейти в пакет theme, где будут находиться файлы для светлой и темной темы приложения, состоящие из специальных стилей, со своим набором атрибутов, позволяющих настраивать внешний вид отдельных компонентов (рисунок 1.4.6).

Эти атрибуты включают цвета, шрифты, размеры, отступы и многие другие параметры, которые определяют визуальное представление элемента. Примеры таких атрибутов включают `colorPrimary` для определения основного цвета темы, `textSize` для размера текста и `background` для фона элемента.

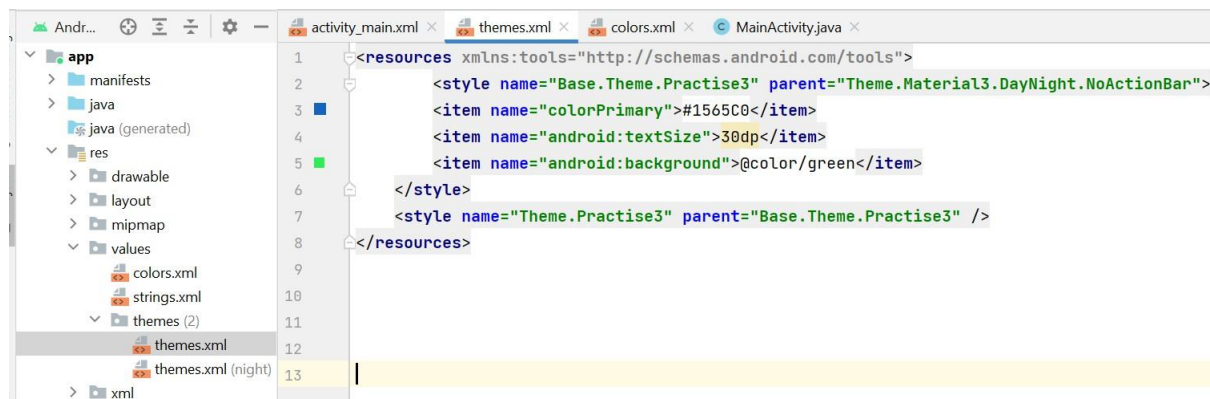


Рисунок 1.4.6 – Код смены темы приложения

Результатом станут измененные параметры для всей темы (рисунок 1.4.7).



Рисунок 1.4.7 – Отображение результата смены темы приложения

Немаловажным элементом в иерархии стилей является параметр `parent`. Он позволяет определить родительский стиль для текущего стиля, наследуя его атрибуты и позволяя модифицировать или расширять их.

Локализация приложения осуществляется похожим образом, отличием является то, что для каждого региона необходимо создавать свой отдельный файл со строками (рисунок 1.4.8). Для этого нужно нажать по пакету `values` правой

кнопкой мыши и в открывшемся окне выбрать параметр Locale и регион, например ru (название файла может отличаться).

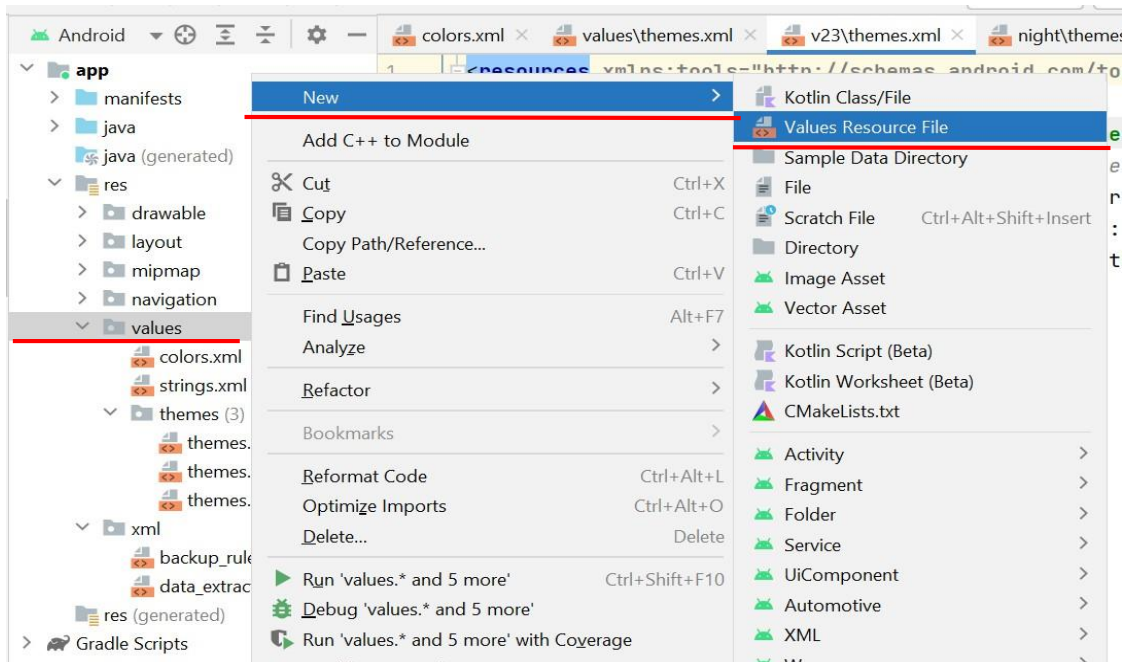


Рисунок 1.4.8 – Создание нового ресурсного файла

В диалоговом окне в левой части Available qualifiers: выбираем пункт Locale и переносим его в правую часть Chosen qualifiers: с помощью кнопки с двумя стрелками вправо. В появившейся третьей колонке выбираем нужные языки, например, русский (рисунок 1.4.9). Поле Directory name автоматически появится нужное название папки.

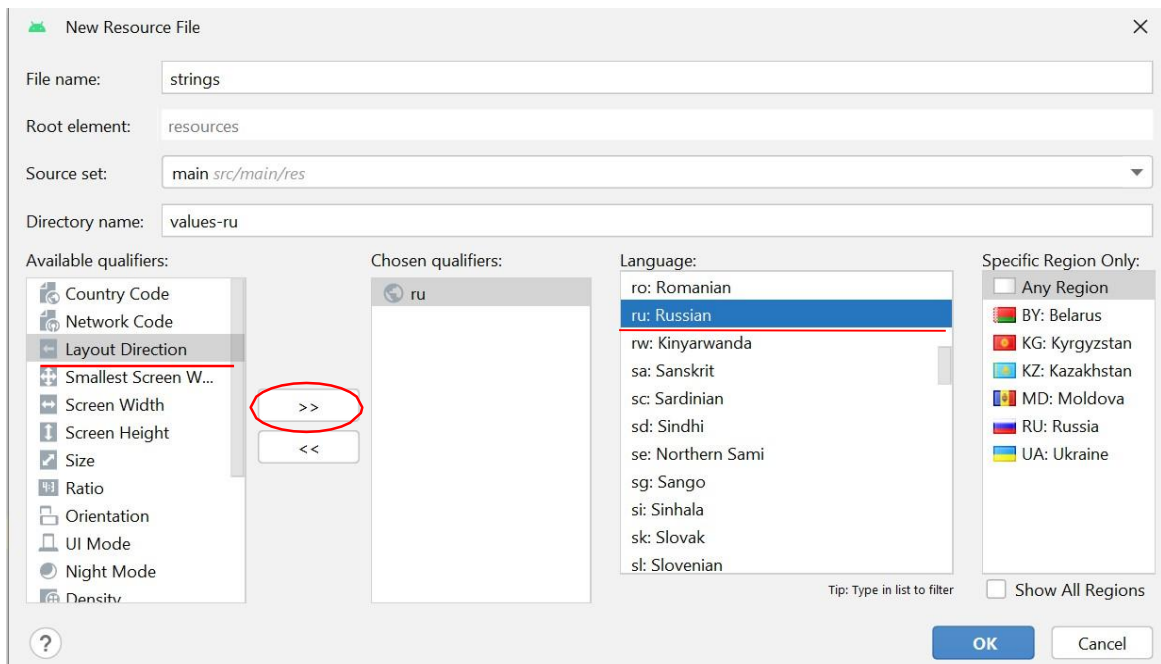


Рисунок 1.4.9 – Выбор русского языка при создании нового ресурсного файла

В результате создания ресурсных файлов в структуре проекта будут добавлены записи в раздел res/values/strings (рисунок 1.4.10).

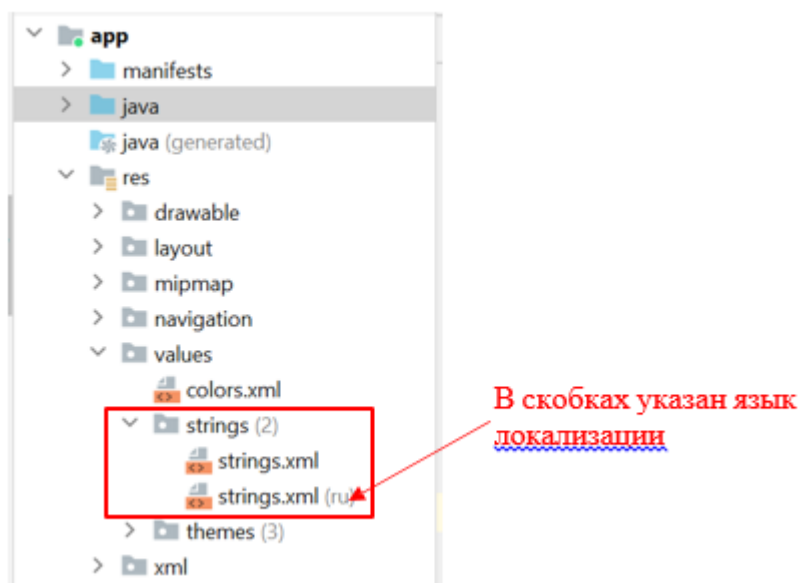


Рисунок 1.4.10 – Расположение нового ресурсного файла

Для корректной работы перевода необходимо созданные ранее строки перенести в новый файл. Таким образом, в данном файле будет содержаться текст для русского региона (рисунки 1.4.11 – 1.4.12).

```
<resources>
    <string name="app_name">Здравствуй мир!</string>
    <!-- TODO: Remove or change this placeholder text -->
    <string name="lbl_login">Логин</string>
    <string name="lbl_password">пароль</string>
    <string name="hint_login">Учетная запись</string>
    <string name="previous">Раньше</string>
    <string name="next">Следующий</string>
</resources>
```

Рисунок 1.4.11 – Код файла values-ru/string.xml

```
<resources>
    <string name="app_name">HelloWorld</string>
    <!-- TODO: Remove or change this placeholder text -->
    <string name="lbl_login">Login</string>
    <string name="lbl_password">Password</string>
    <string name="hint_login">Account</string>
    <string name="next">Next</string>
    <string name="previous">Previous</string>
</resources>
```

Рисунок 1.4.12 – Код файла values/string.xml

Если запустить приложение, то локализация будет работать (рисунок 1.4.13).

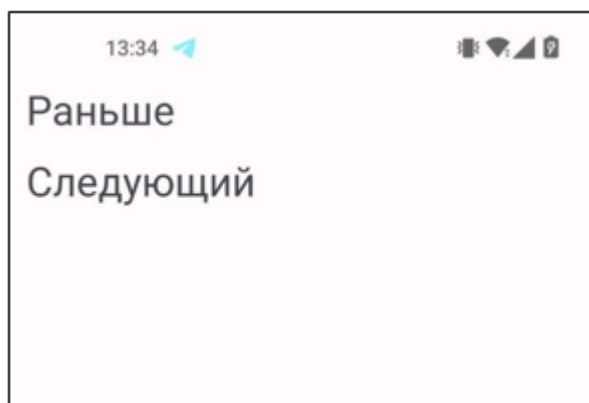


Рисунок 1.4.14 – Отображение локализации в итоговом виде приложения

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Создание проекта

В данной практической работе в качестве шаблона выбираем Empty Views Activity (рисунок 2.1.1).

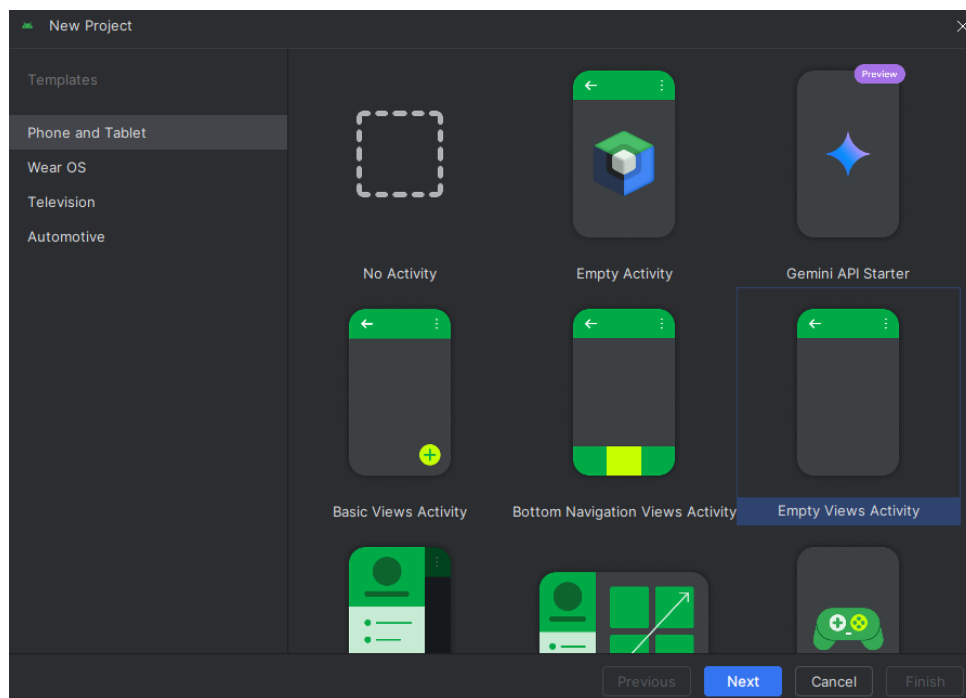


Рисунок 2.1.1 – Выбор шаблона итогового приложения

Далее назовём проект согласно номеру практической работы (рисунок 2.1.2).

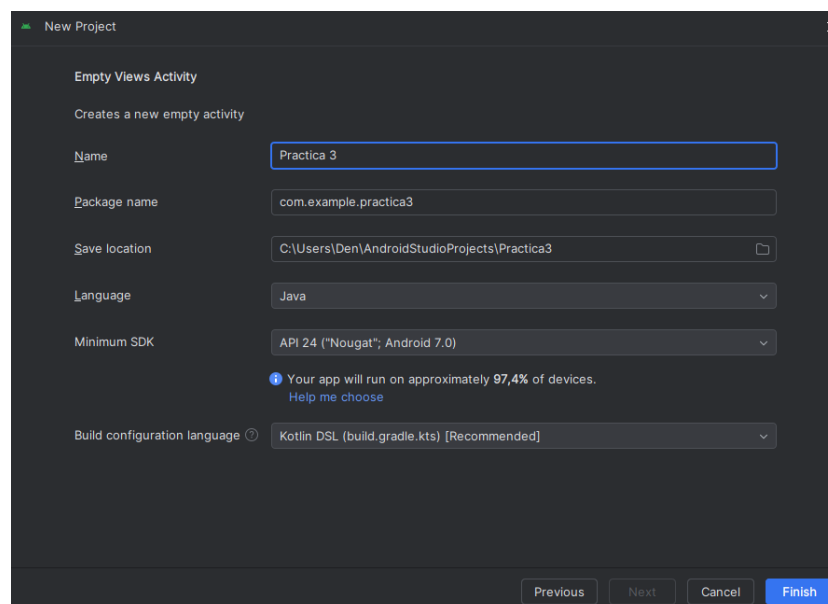


Рисунок 2.1.2 – Настройка итогового приложения

2.2 ConstraintLayout

В качестве первого примера выполнения задания создадим активность с применением ConstraintLayout (рисунок 2.2.1).

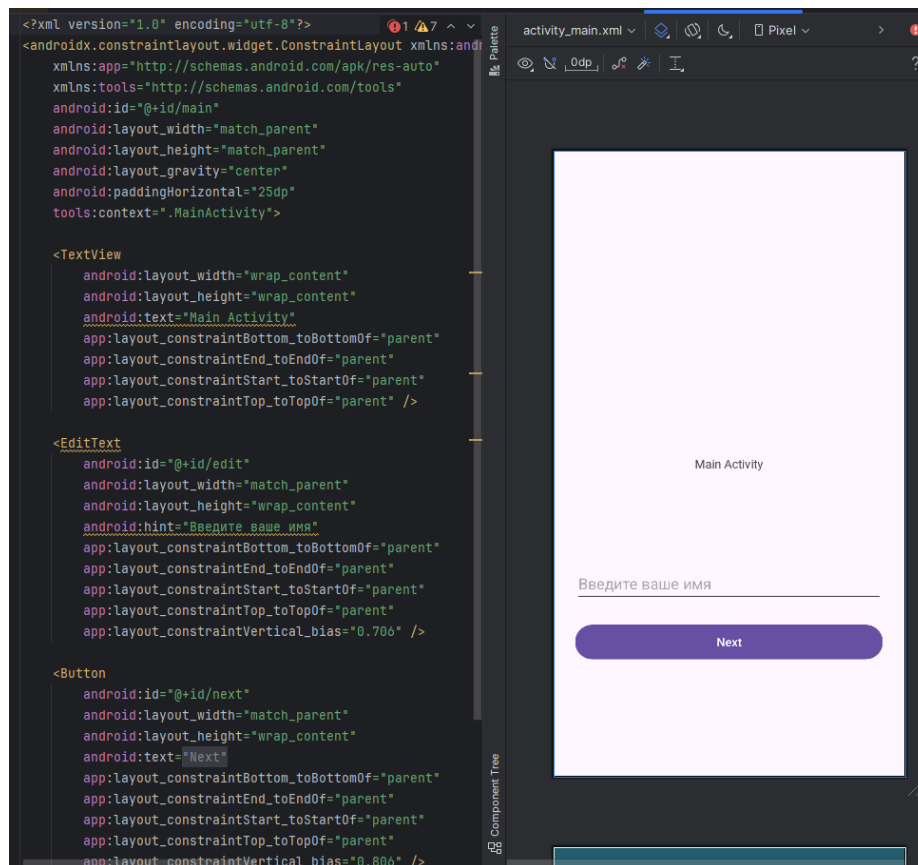


Рисунок 2.2.1 – Кодовое и графическое отображение activity с контейнером ConstraintLayout

На рисунке 2.2.1 такие атрибуты, как `android:layout_width` и `android:layout_height` имеют одинаковые значения потому, что так достигается понятный и приятный UI. А атрибут `app:layout_constraintVertical_bias` присвоен элементам `EditText` и `Button` с целью определить их местоположение.

Также была сделана возможность перехода на следующую страницу при помощи элемента `Button` (рисунок 2.2.2).

```

1 package com.example.practica3;
2
3 > import ...
4
15 <?> public class MainActivity extends AppCompatActivity {
16
17     @Override
18     protected void onCreate(Bundle savedInstanceState) {
19         super.onCreate(savedInstanceState);
20         EdgeToEdge.enable(this);
21         setContentView(R.layout.activity_main);
22         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
23             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
24             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
25             return insets;
26         });
27         Button nextButton = findViewById(R.id.next);
28         nextButton.setOnClickListener(new View.OnClickListener() {
29             @Override
30             public void onClick(View v) {
31                 onNextActivity(v);
32             }
33         });
34     }
35     ! usage
36     public void onNextActivity(View view){
37         EditText edit = findViewById(R.id.edit);
38         String name = edit.getText().toString();
39
40         MyObject myObject = new MyObject(name, age, 0);
41         Intent intent = new Intent(packageContext, Linear.class);
42         intent.putExtra("name", myObject.getName());
43         startActivity(intent);
44     }

```

Рисунок 2.2.2 – Код, реализующий логику перехода на другую activity

При этом на рисунке 2.2.2 можно заметить, как передаётся строковый атрибут в метод putExtra, который будет считан последующей activity.

2.3 LinearLayout

Данный контейнер позволяет довольно просто расположить необходимые элементы по всему файлу разметки (рисунок 2.3.1).

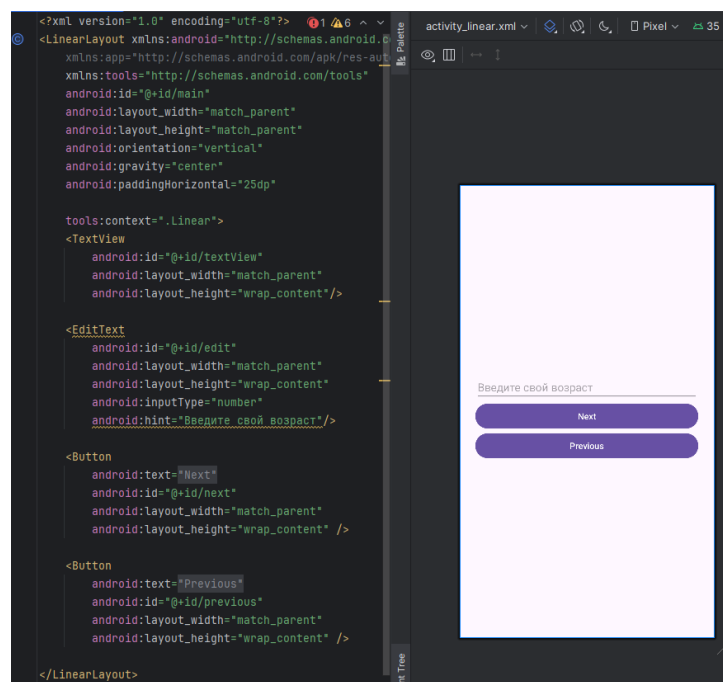
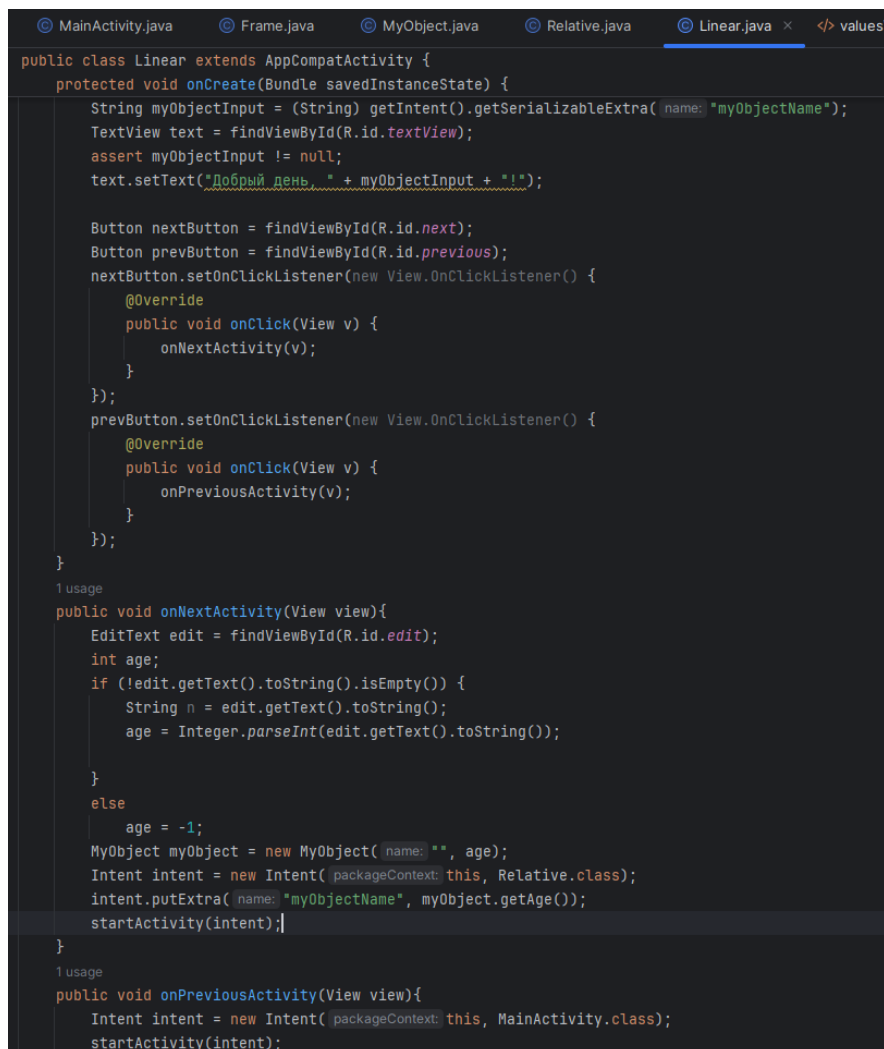


Рисунок 2.3.1 - Кодовое и графическое отображение activity с контейнером LinearLayout

В данном файле разметки была добавлена кнопка перехода на предыдущую activity с сохранением введённых значений.

Также для этого файла разметки был написан код, реализующий логику работу его элементов (рисунок 2.3.2).



```
public class Linear extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        String myObjectInput = (String) getIntent().getSerializableExtra( name: "myObjectName");
        TextView text = findViewById(R.id.textView);
        assert myObjectInput != null;
        text.setText("Добрый день, " + myObjectInput + "!");

        Button nextButton = findViewById(R.id.next);
        Button prevButton = findViewById(R.id.previous);
        nextButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                onNextActivity(v);
            }
        });
        prevButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                onPreviousActivity(v);
            }
        });
    }

    1 usage
    public void onNextActivity(View view){
        EditText edit = findViewById(R.id.edit);
        int age;
        if (!edit.getText().toString().isEmpty()) {
            String n = edit.getText().toString();
            age = Integer.parseInt(edit.getText().toString());
        }
        else
            age = -1;
        MyObject myObject = new MyObject( name: "", age);
        Intent intent = new Intent( packageContext: this, Relative.class);
        intent.putExtra( name: "myObjectName", myObject.getAge());
        startActivity(intent);
    }

    1 usage
    public void onPreviousActivity(View view){
        Intent intent = new Intent( packageContext: this, MainActivity.class);
        startActivity(intent);
    }
}
```

Рисунок 2.3.2 - Код, реализующий логику перехода на другую activity

При этом на рисунке 2.3.2 отображён код, реализующий переход на предыдущую страницу в методе onPreviousActivity.

2.4 RelativeLayout

Данный контейнер позволяет располагать элементы интерфейса относительно друг друга или относительно родительского контейнера. И в нём также будут размещены некоторые элементы интерфейса (2 кнопки, 1 EditText и 1 TextView) (рисунки 2.4.1 – 2.4.2).

```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:background="#168BC34A"
    tools:context=".Relative">

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"/>

    <EditText
        android:id="@+id/edit"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_alignParentEnd="true"
        android:layout_marginTop="520dp"
        android:layout_marginEnd="0dp"
        android:hint="Введите своё любимое хобби" />

    <Button
        android:id="@+id/next"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_alignParentEnd="true"
        android:layout_marginTop="584dp"
        android:layout_marginEnd="0dp"
        android:text="Next" />

```

Рисунок 2.4.1 – Часть 1 кодового и графического отображения activity с контейнером RelativeLayout

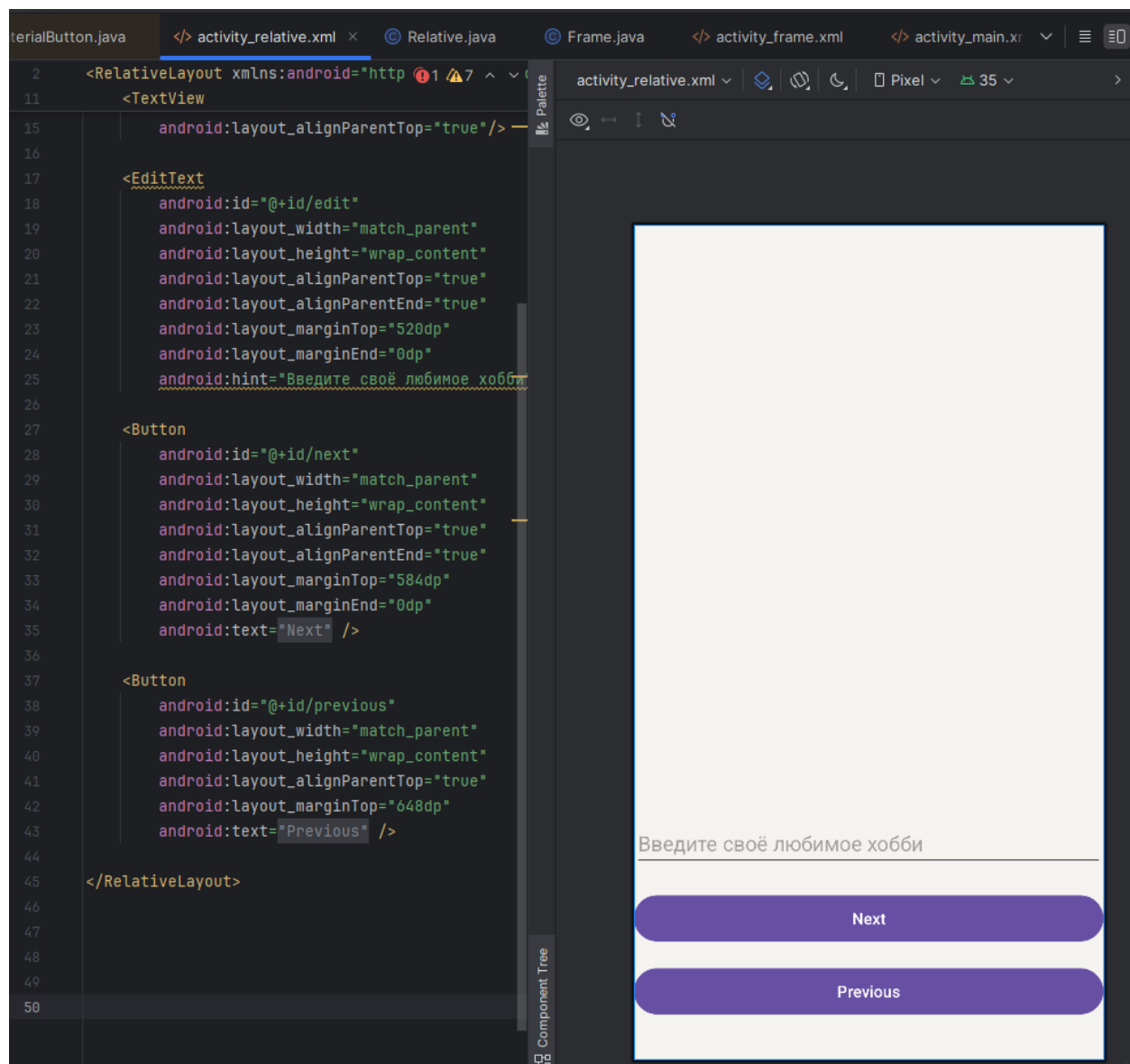


Рисунок 2.4.2 – Часть 2 кодового и графического отображения activity с контейнером RelativeLayout

Добавляя такие атрибуты, как `android:layout_alignParentTop` и `android:layout_alignParentEnd` мы можем расположить элементы интерфейса по своему усмотрению, используя атрибут `android:layout_marginTop`, задающий расстояние от верха экрана до самого элемента.

При этом был реализован переход на другие страницы в виде кода (рисунок 2.4.3).

```

rRU\strings.xml  MainActivity.java  Frame.java  MyObject.java  Relative.java  x  </> values\st

public class Relative extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {

        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_relative);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
        int myObjectInput = (int) getIntent().getSerializableExtra( name: "myObjectName");
        TextView text = findViewById(R.id.textView);
        text.setText("Вам " + myObjectInput + " лет");

        Button nextButton = findViewById(R.id.next);
        Button prevButton = findViewById(R.id.previous);
        nextButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                onNextActivity(v);
            }
        });
        prevButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                onPreviousActivity(v);
            }
        });
    }
    1 usage
    public void onNextActivity(View view){
        EditText edit = findViewById(R.id.edit);
        String name = edit.getText().toString();

        MyObject myObject = new MyObject(name, age: 0);
        Intent intent = new Intent( packageContext: this, Frame.class);
        intent.putExtra( name: "myObjectName", myObject.getName());
        startActivity(intent);
    }
    1 usage
    public void onPreviousActivity(View view){
        Intent intent = new Intent( packageContext: this, Linear.class);
        startActivity(intent);
    }
}

```

Рисунок 2.4.3 - Код, реализующий логику перехода на другую activity

2.5 FrameLayout

Данный контейнер идеально подойдёт для нескольких элементов — это как раз наш случай, ведь в данном файле разметки мы отобразим всего 2 элемента интерфейса: TextView и Button (рисунок 2.5.1).

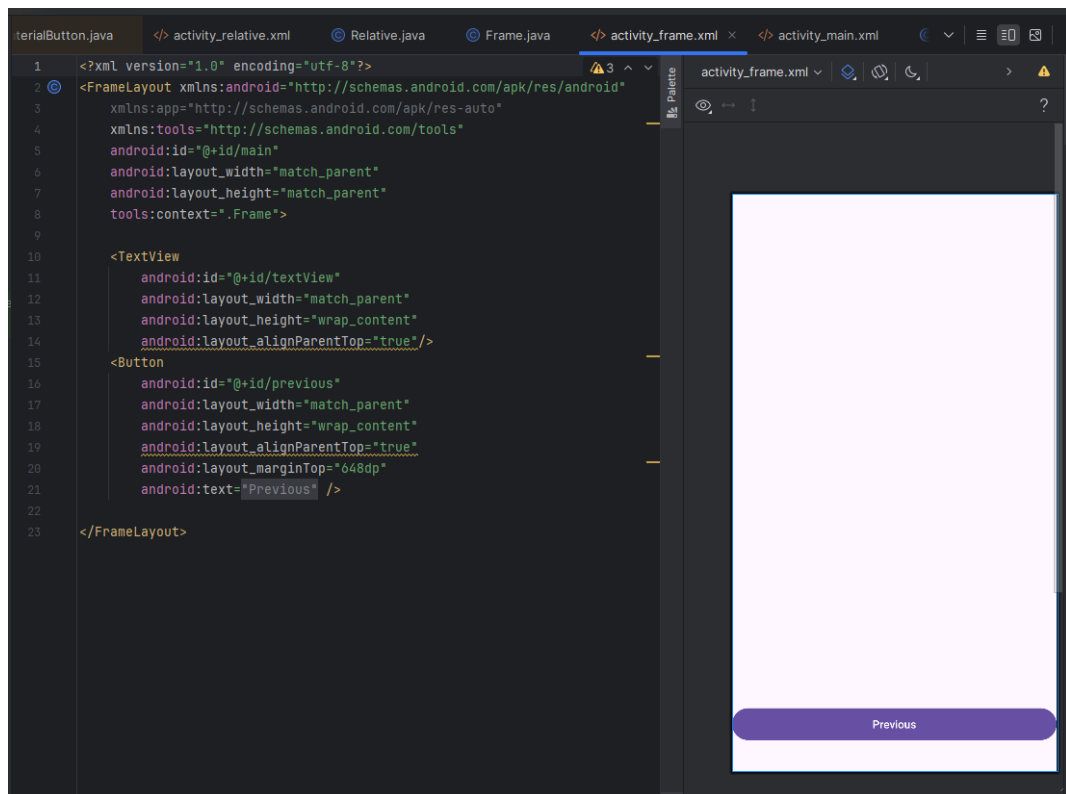


Рисунок 2.5.1 - Кодовое и графическое отображение activity с контейнером FrameLayout

При этом в данной activity также была реализована логика перемещения на другую activity – предыдущую (рисунок 2.5.2).

```

15 public class Frame extends AppCompatActivity {
16
17     @Override
18     protected void onCreate(Bundle savedInstanceState) {
19         super.onCreate(savedInstanceState);
20         EdgeToEdge.enable(this);
21         setContentView(R.layout.activity_frame);
22         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
23             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
24             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
25             return insets;
26         });
27         Button prevButton = findViewById(R.id.previous);
28         prevButton.setOnClickListener(new View.OnClickListener() {
29             @Override
30             public void onClick(View v) {
31                 onPreviousActivity(v);
32             }
33         });
34
35         String myObjectInput = (String) getIntent().getSerializableExtra("myObjectName");
36         TextView text = findViewById(R.id.textView);
37         assert myObjectInput != null;
38         text.setText("Ваше любимое хобби - " + myObjectInput);
39     }
40
41     1 usage
42     public void onPreviousActivity(View view){
43         Intent intent = new Intent(packageContext, this, Relative.class);
44         startActivity(intent);
45     }

```

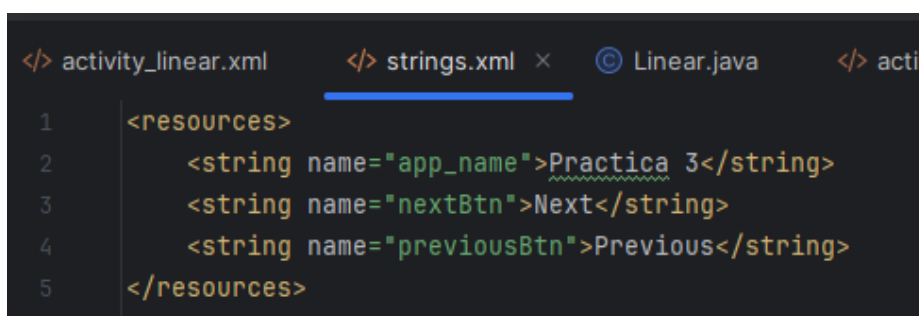
Рисунок 2.5.2 - Код, реализующий логику перехода на другую activity

2.6 Добавление ресурсных файлов

В данный проект будут добавлены следующие ресурсы:

- строковые;
- размерные;
- цветовые;
- drawable.

Для начала добавим строковые ресурсы (рисунок 2.6.1).



```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Practica 3</string>
    <string name="nextBtn">Next</string>
    <string name="previousBtn">Previous</string>
</resources>
```

Рисунок 2.6.1 – Файл строковых ресурсов

Были добавлены ресурсы для таких имён, как “nextBtn” и “previousBtn”. Данные строковые ресурсы использовались при определении текста, показываемого на кнопках перехода от одной activity к другой.

Далее идёт добавление размерного ресурса. Однако также пришлось создать дополнительный ресурсный файл под названием `dimens.xml`, в котором и будут храниться все размерные ресурсы (рисунок 2.6.2).

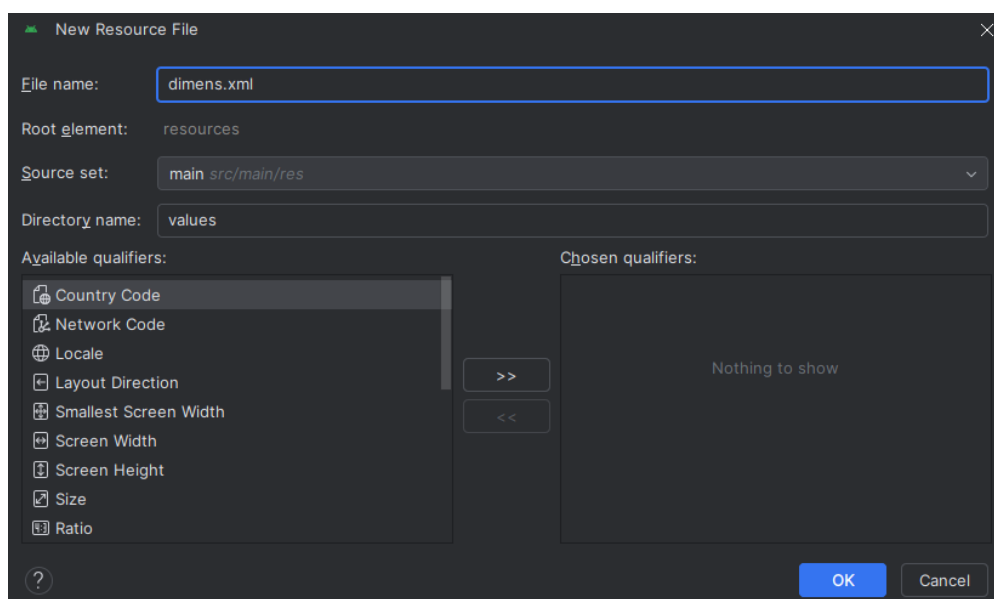


Рисунок 2.6.2 – Добавление размерного ресурсного файла

И теперь добавим размерный ресурс (рисунок 2.6.3).

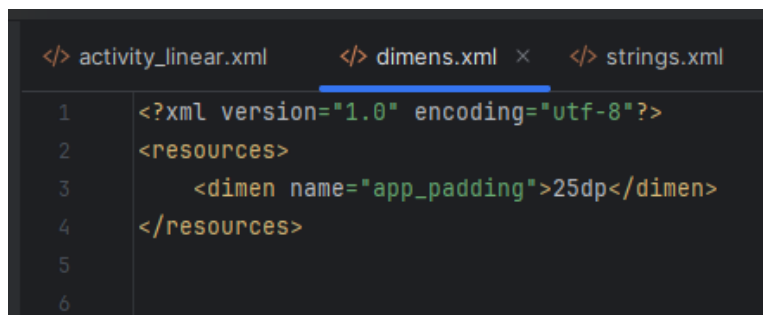


Рисунок 2.6.3 – Добавление размерного ресурса

Данный ресурс мог бы использоваться для определения расстояния между элементами интерфейса и их контейнером.

За размерным ресурсом следуют цветовые (рисунок 2.6.4).

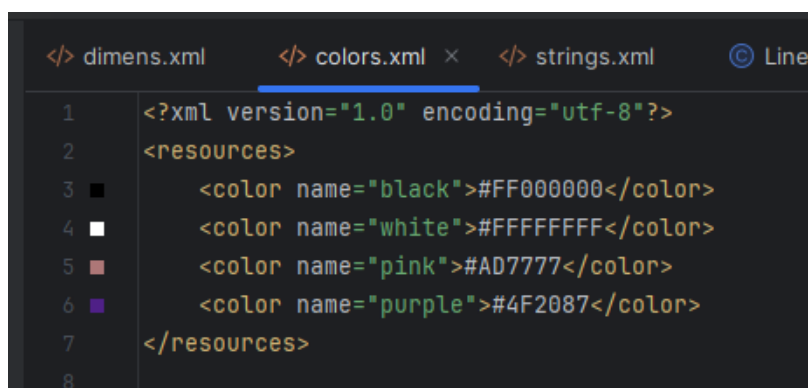


Рисунок 2.6.4 – Добавление цветовых ресурсов

Были добавлены цветовые ресурсы на такие имена, как “pink” и “purple”. Они будут использованы в дальнейшем при изменении темы приложения.

И последний ресурс – drawable. Для начала нам необходимо перенести копируемое изображение в папку drawables (рисунок 2.6.5).

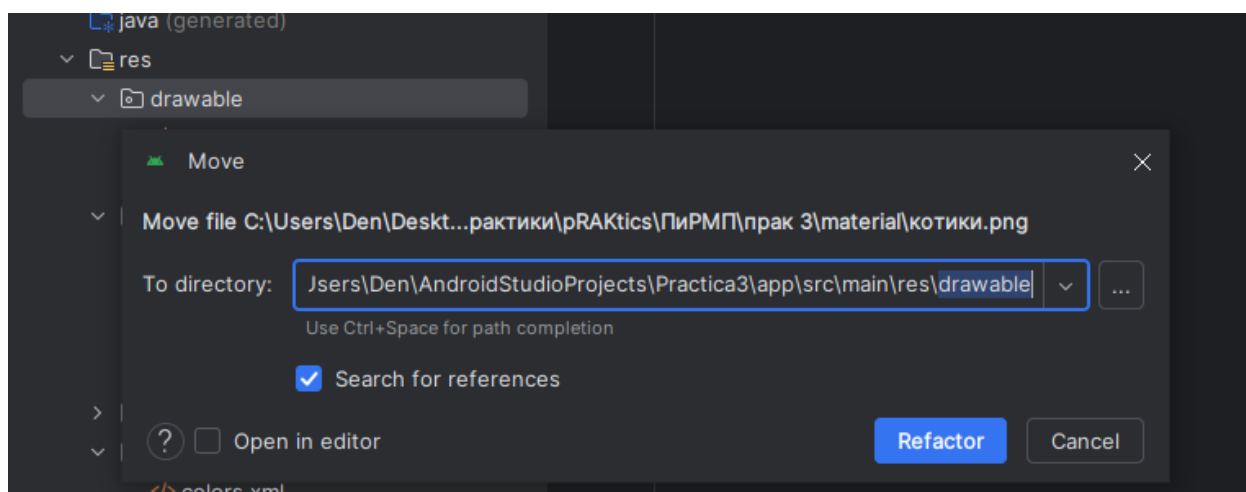


Рисунок 2.6.5 – Добавление drawable ресурса

А затем убедимся в том, что изображение отображается корректно (рисунок 2.6.6).

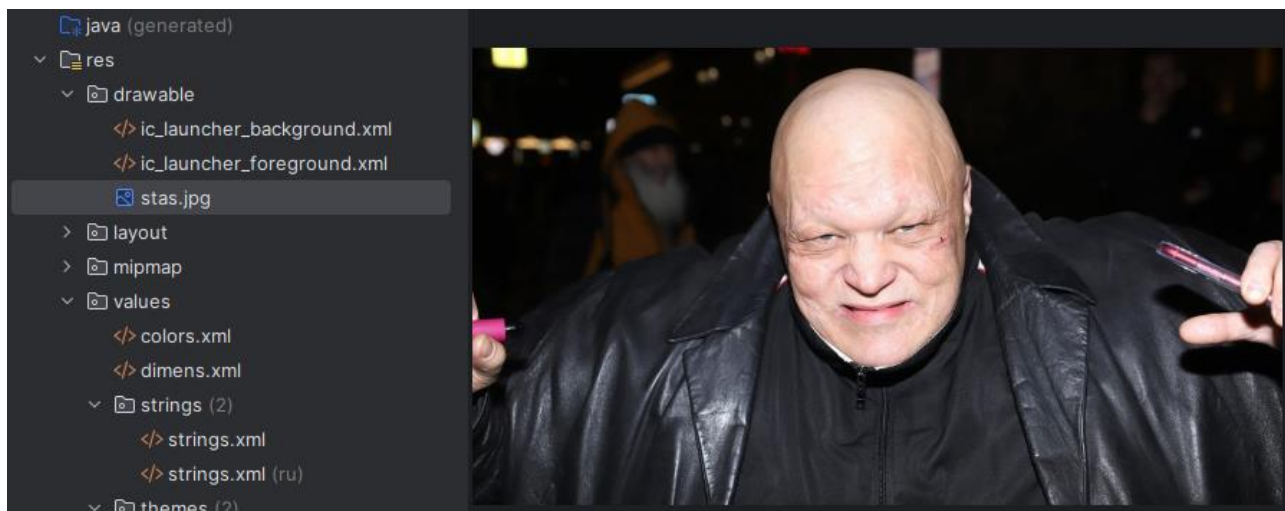


Рисунок 2.6.6 – Отображение добавленного drawable ресурса

2.7 Изменение темы

Для изменения темы надо перейти в файл themes.xml и внести в него необходимые изменения (рисунок 2.7.1).

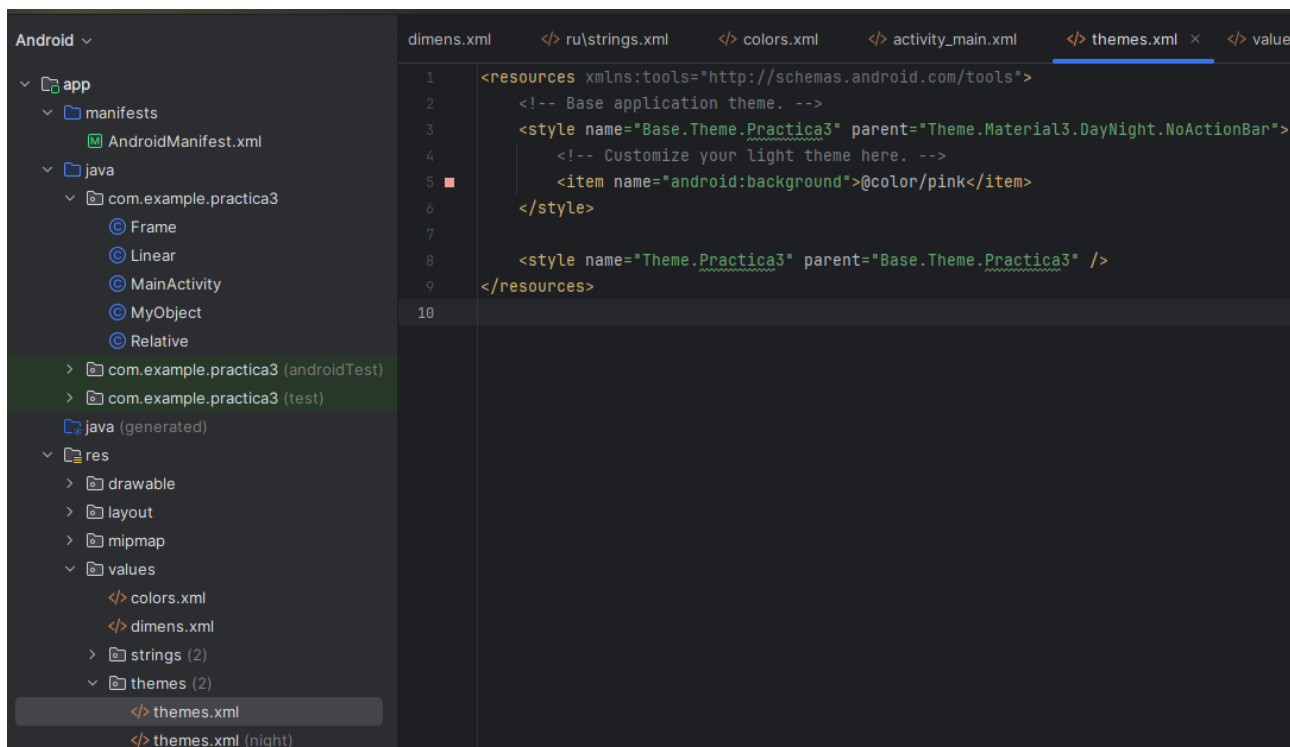


Рисунок 2.7.1 – Изменение темы приложения

Строка `<item name="android:background">@color/pink</item>` была добавлена. Данная строка задаёт задний фон приложения как розовый цвет. Также добавим в приложение новый drawable ресурс – stas.png (рисунок 2.7.2).

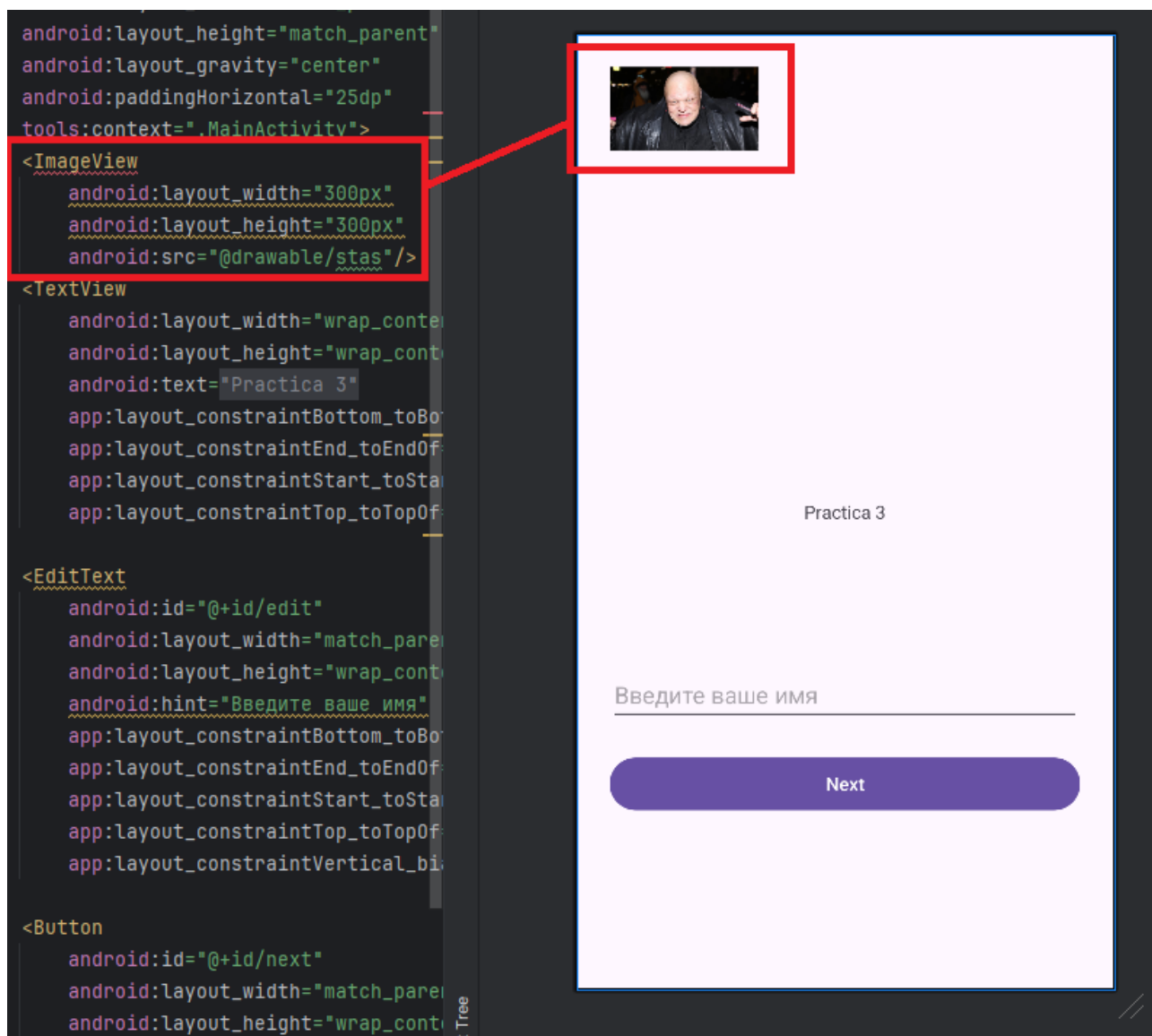


Рисунок 2.7.2 – Добавление drawable ресурса в итоговое приложение

Изменив тему, убедимся в том, что приложение работает корректно (рисунок 2.7.3).

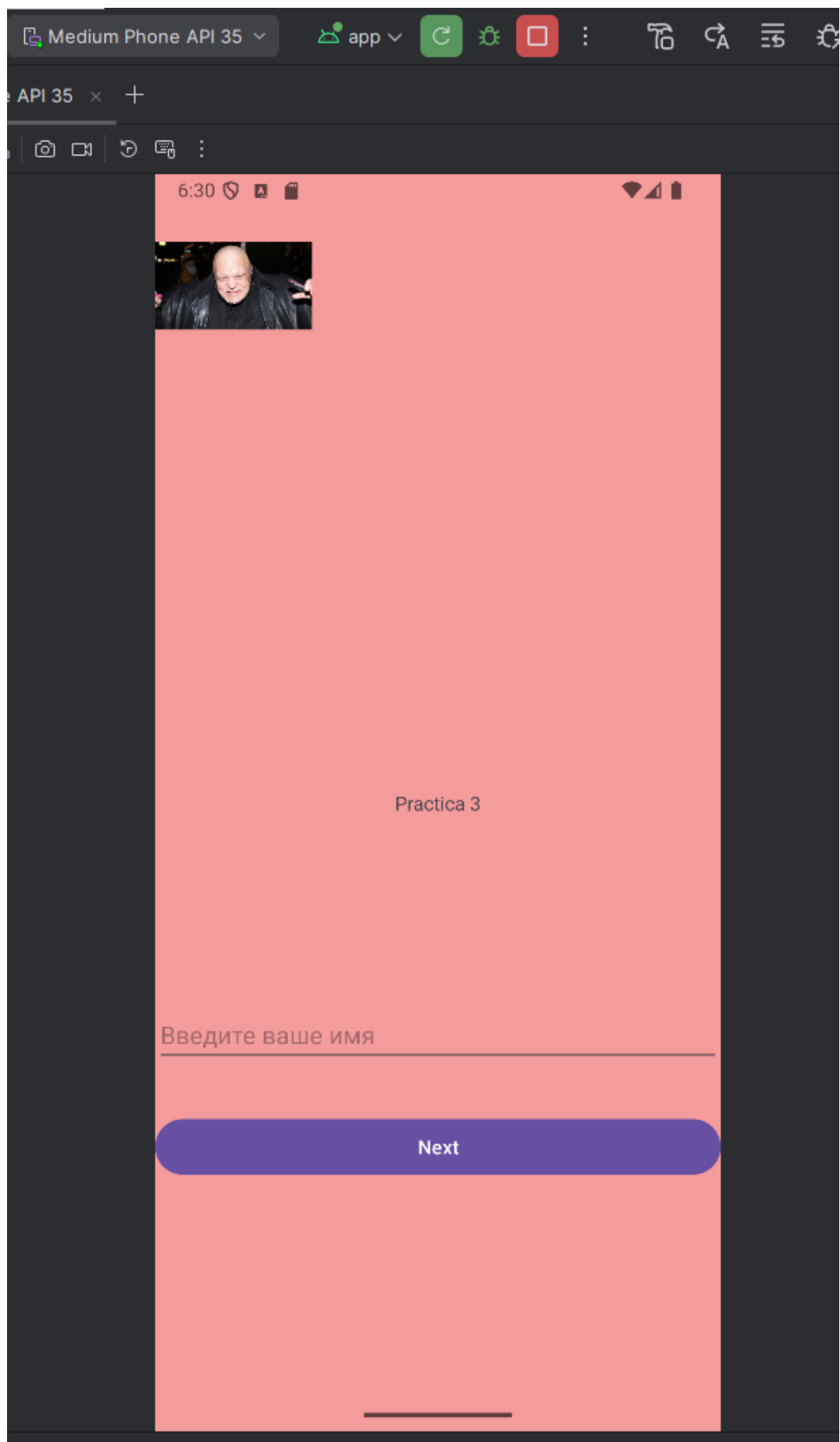


Рисунок 2.7.3 – Изменённый вид приложения

2.8 Локализация

Для добавления локализации необходимо добавить новый строковый ресурсный файл strings.xml. Для этого необходимо перейти в папку res, нажать правую кнопку мыши на папку strings, далее нажать на New, а после на Values Resource File (рисунок 2.8.1).

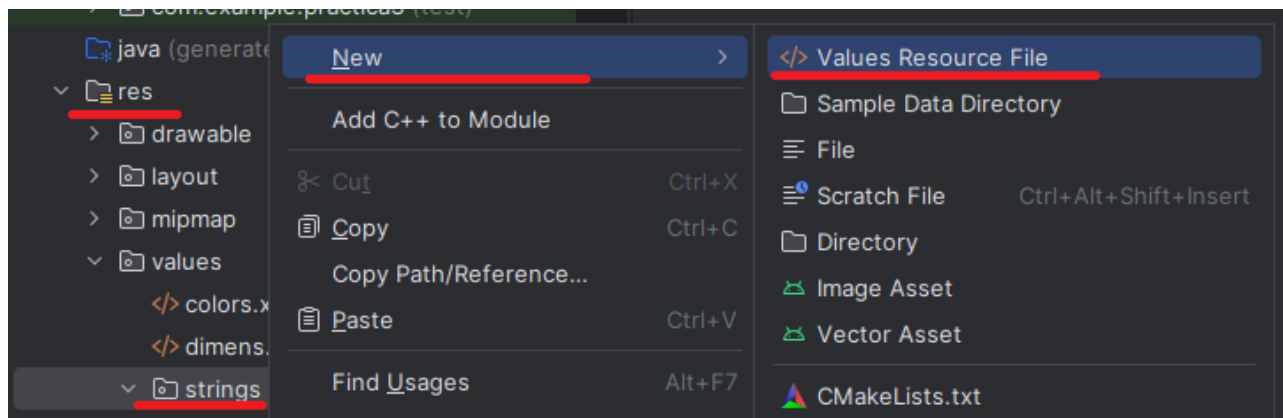


Рисунок 2.8.1 – Добавление нового строкового ресурсного файла

После этого необходимо выбрать локализуемый язык: русский – в данном случае (рисунок 2.8.2).

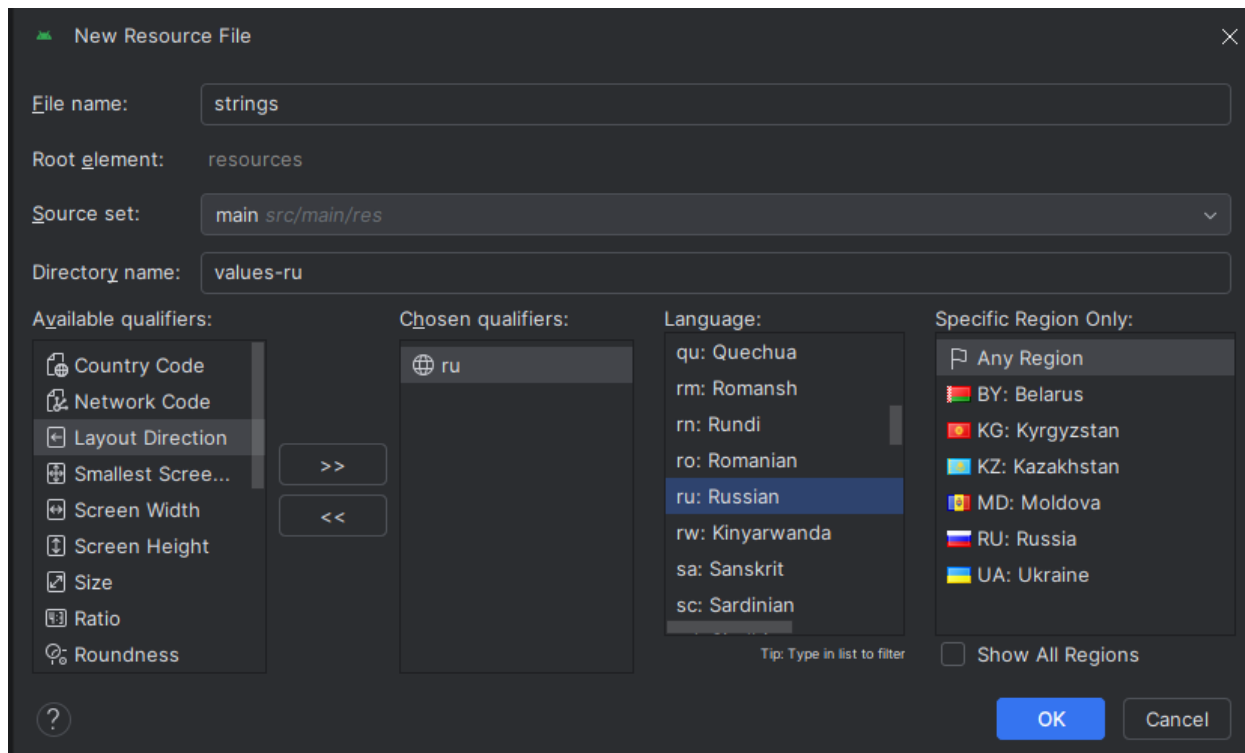


Рисунок 2.8.2 – Выбор локализуемого языка

Теперь перейдём в созданный файл, перенеся в него значения из ресурсного файла strings.xml, и поменяем их значения под локализуемый язык (рисунок 2.8.3).

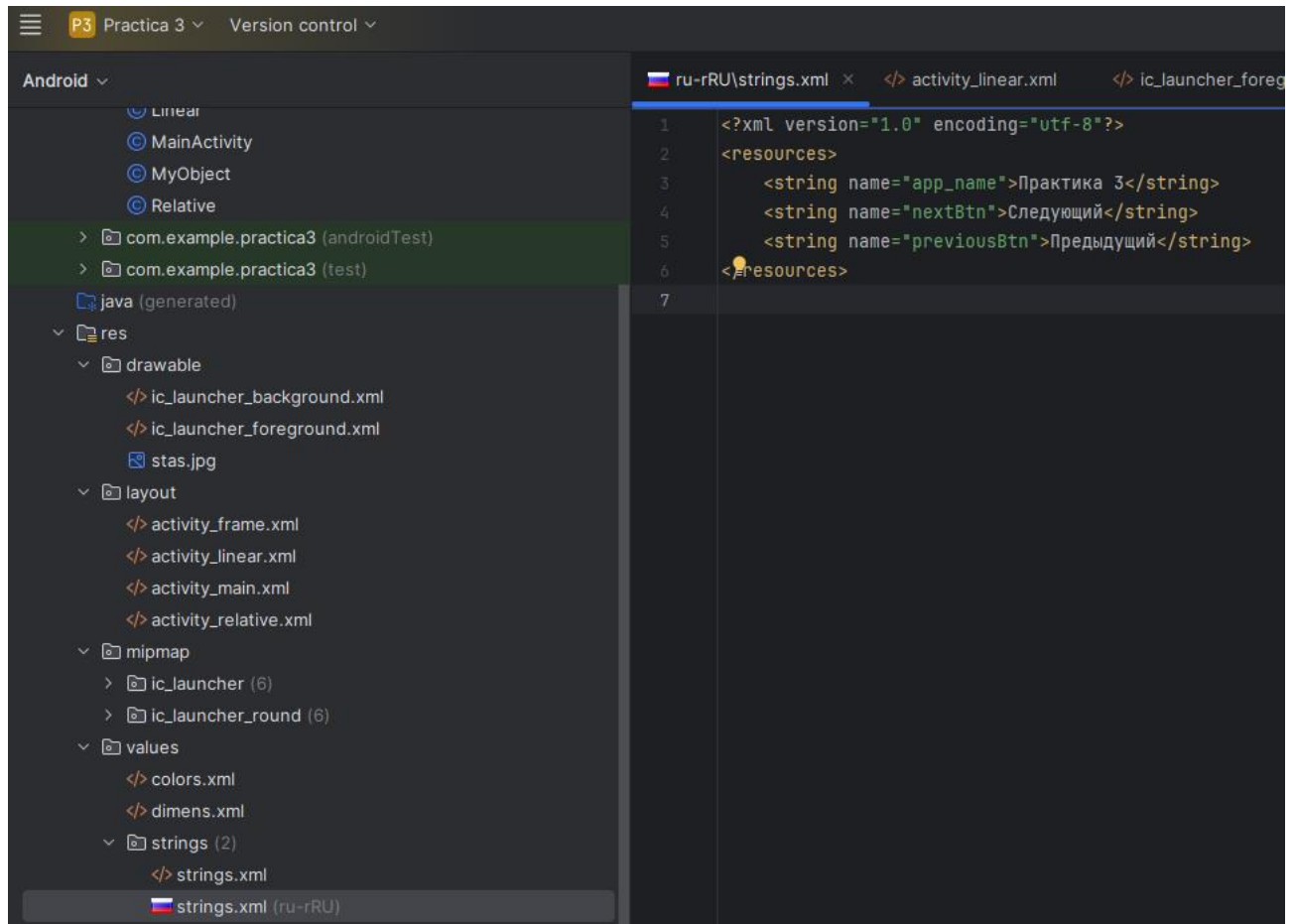


Рисунок 2.8.3 – Локализованный строковый ресурсный файл strings.xml

Теперь убедимся, что в текущем регионе язык заменится на русский (рисунок 2.8.4).

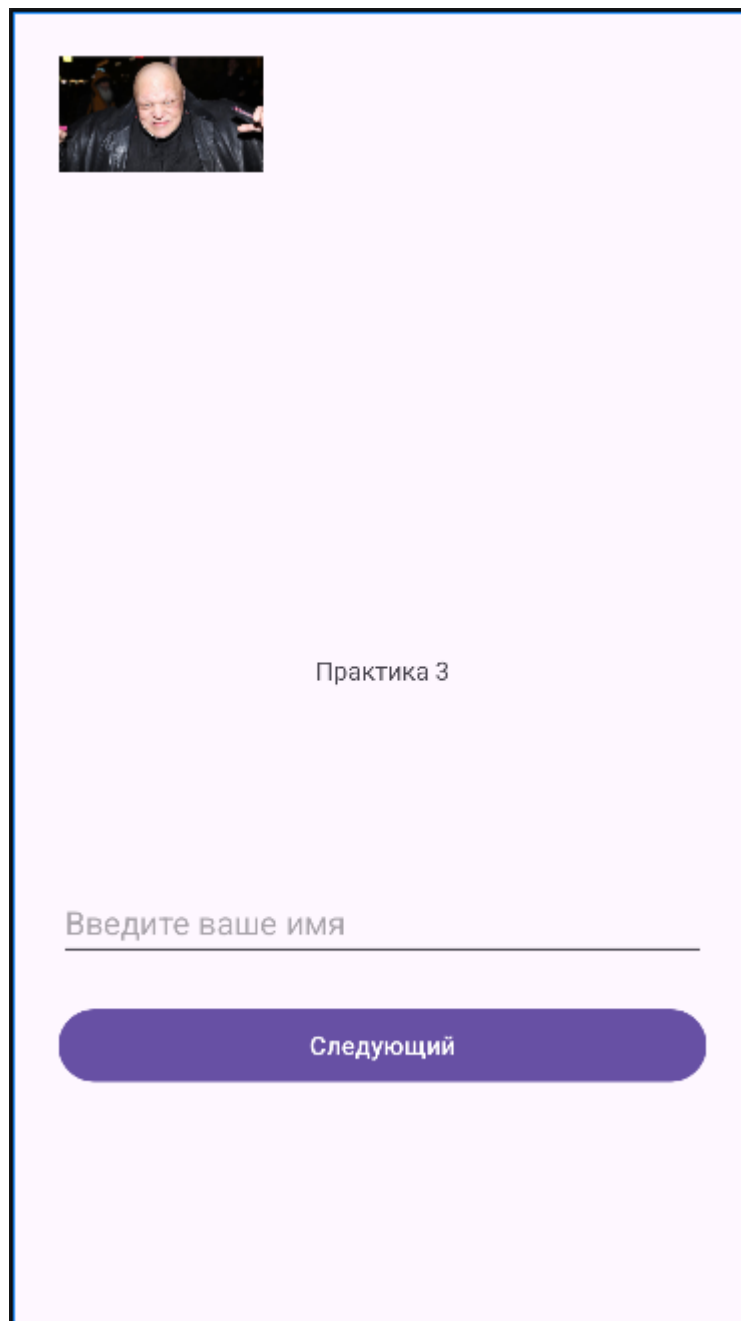


Рисунок 2.8.4 – Локализованный вид приложения

Далее идёт демонстрация рабочего приложения со стандартной темой (рисунки 2.8.5-2.8.8).

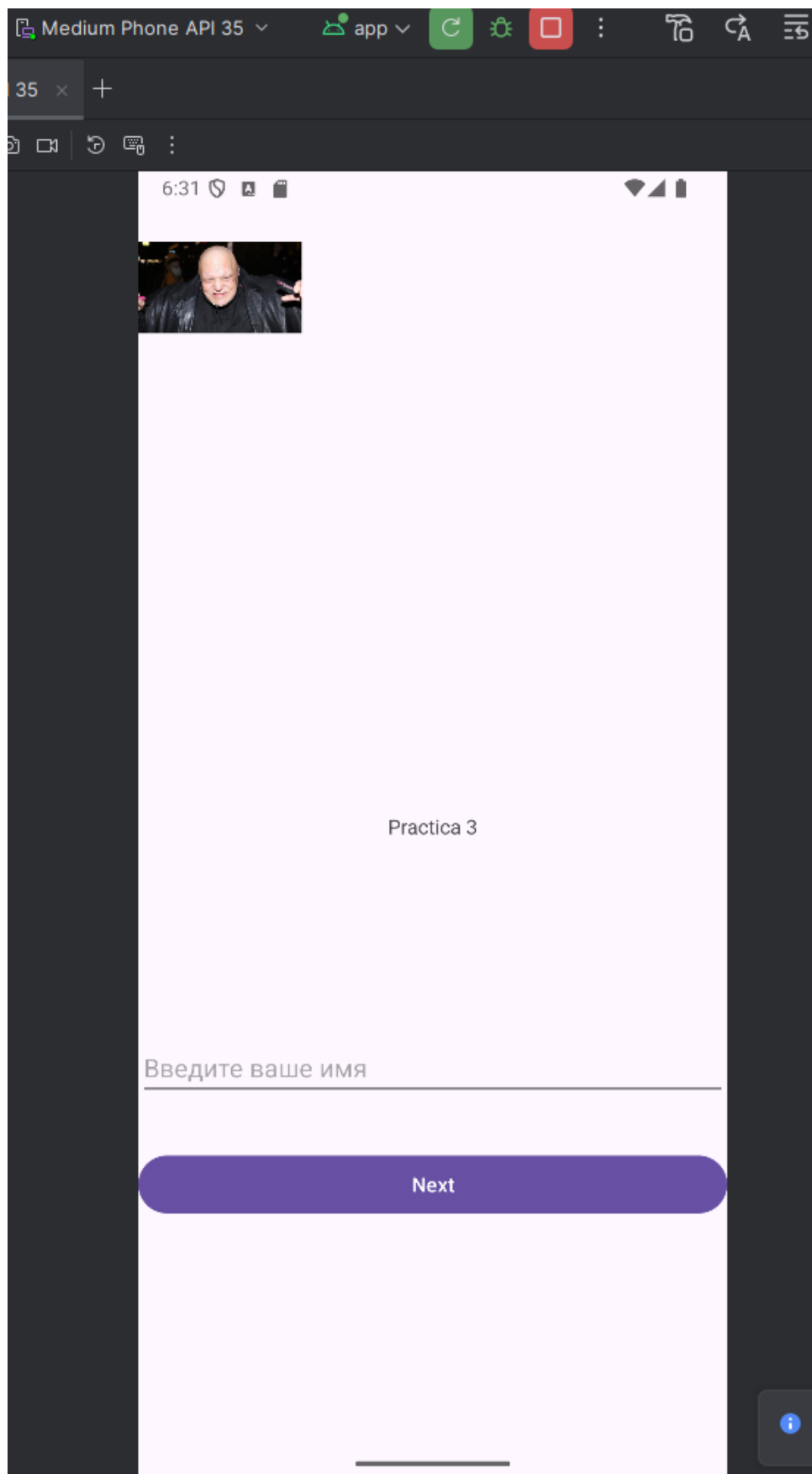


Рисунок 2.8.5 – Показ рабочего приложения часть 1

Рисунок 2.8.5 отображает несколько изменённую приветственную страницу рабочего приложения, где от пользователя требуется ввести его имя.

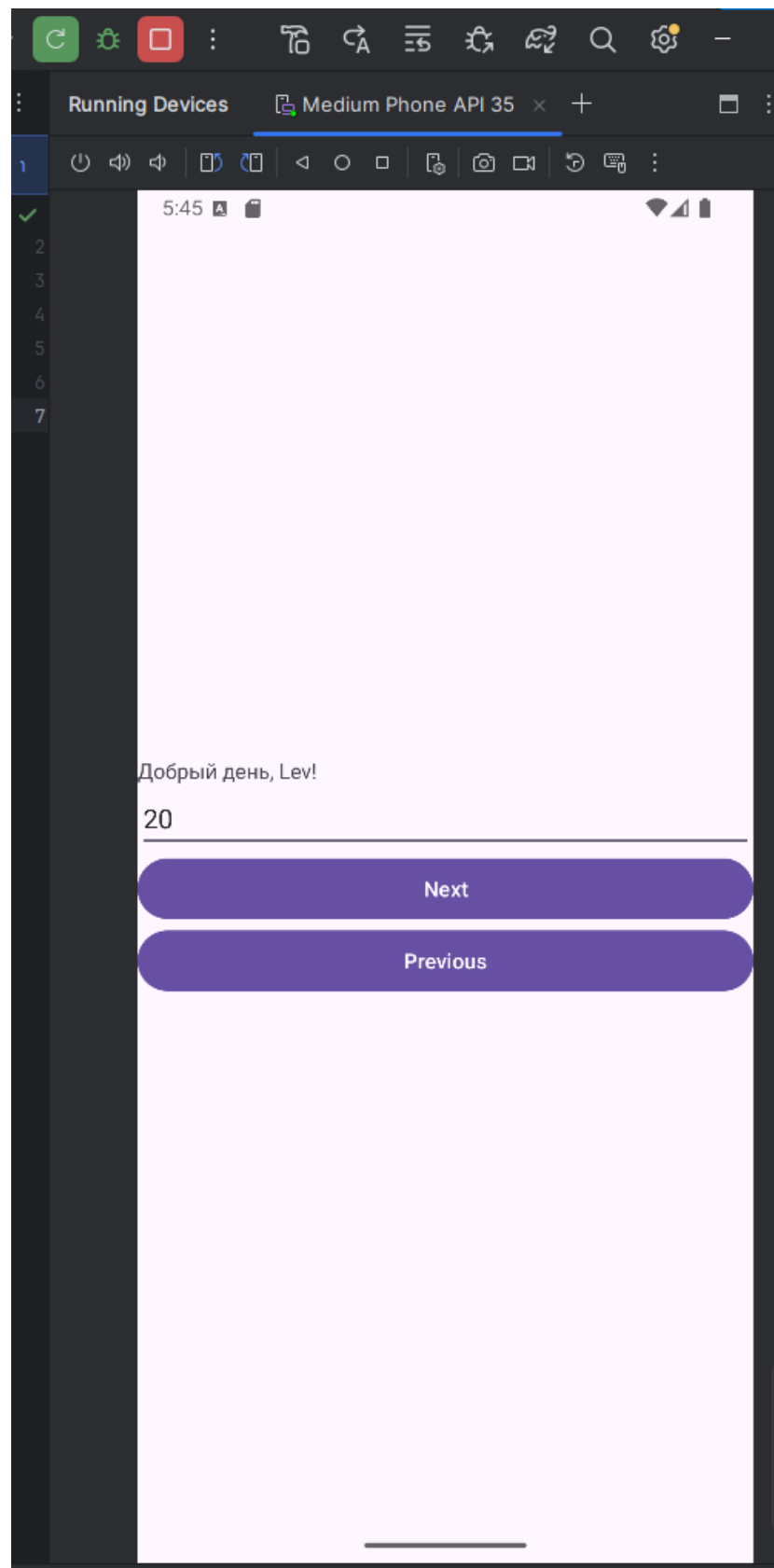


Рисунок 2.8.6 – Показ рабочего приложения часть 2

Далее следует рисунок 2.8.6, отображающий вторую страницу рабочего приложения, на которой от пользователя требуется ввести его возраст.

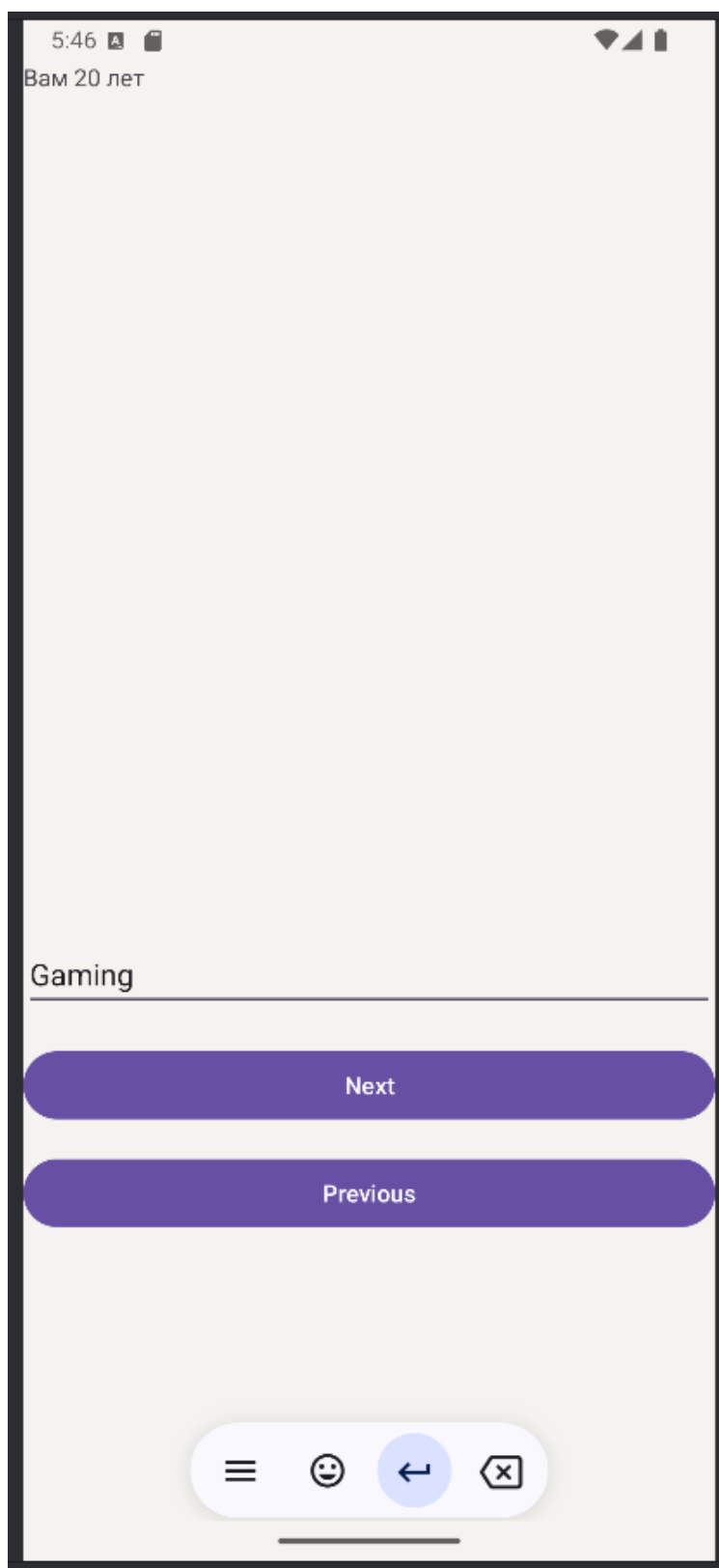


Рисунок 2.8.7 – Показ рабочего приложения часть 3

Третья страница итогового приложения требует от пользователя ввода его любимого хобби.

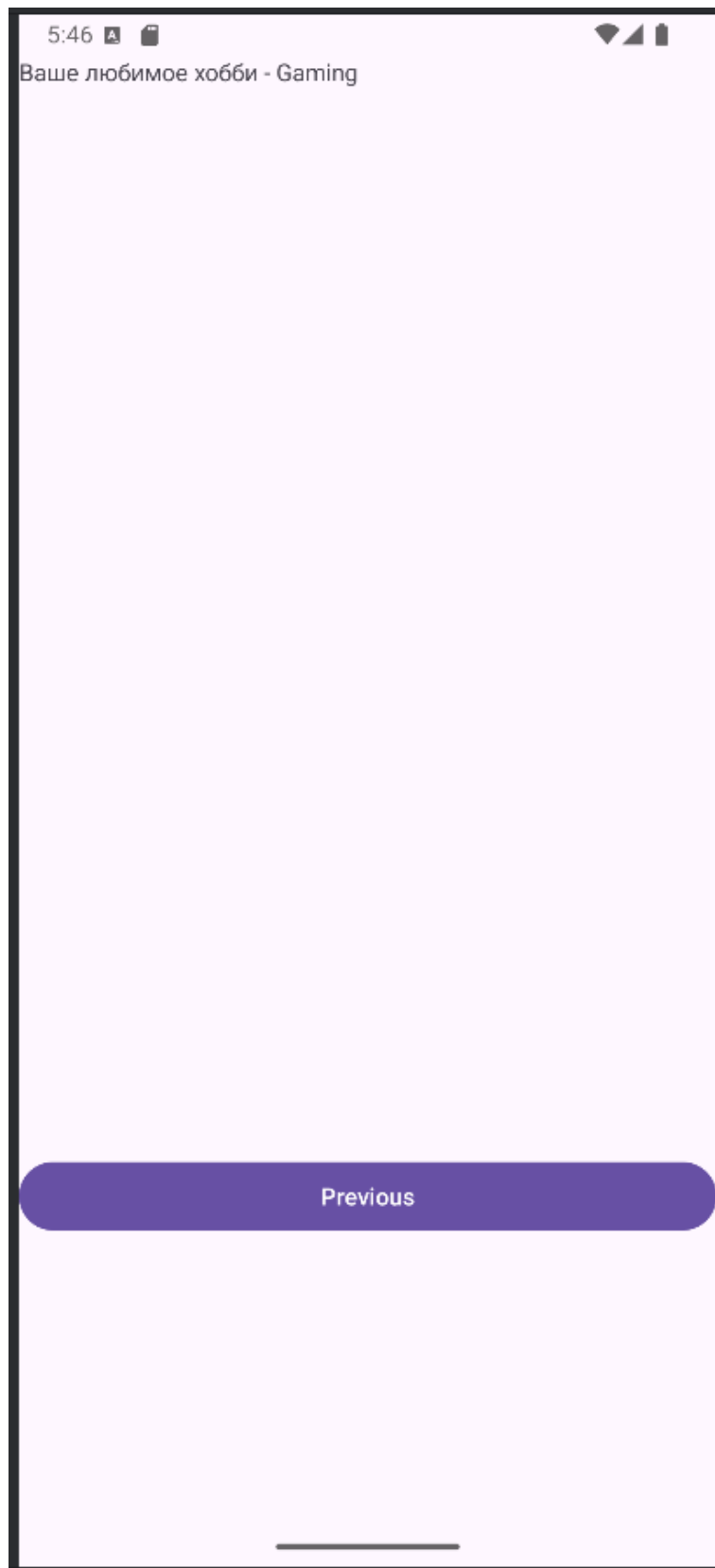


Рисунок 2.8.8 – Показ рабочего приложения часть 4

И последняя страница приложения – отображает введённый ранее пользователем ответ на его любимое хобби. В ходе выше показанных рисунков можно было убедиться, что приложение работает корректно.

ЗАКЛЮЧЕНИЕ

В ходе данной работы было проведено ознакомление с такими понятиями, как передача сложных объектов между activities, BackStack, контейнеры и ресурсы в Android. Пройденный материал был закреплён практическим заданием.